

Neuentwicklung einer Software für die Schadenserfassung an Straßeneinrichtungen

Ausbildungsberuf: Fachinformatiker für Anwendungsentwicklung

Projektzeitraum: 01.03.2026 – 23.04.2026



1

Ausbildungsbetrieb:

Landesbetrieb Mobilität
Rheinland-Pfalz
Friedrich-Ebert-Ring 14-20
56068 Koblenz

Auszubildender:

Timo Funk

Inhalt

1. Projektbeschreibung	1
1.1 Einleitung	1
1.2 Projektumfeld	1
1.3 Projektziel	1
1.4 Projektzeitraum	1
1.5 Projektbetreuung und Entwicklungsumgebung	1
1.6 Anforderungsanalyse mit Auftraggeber	2
1.7 Änderung gegenüber dem Projektantrag	2
2. Projektplanung	2
2.1 Ist-Analyse	2
2.2 Soll-Konzept	3
2.3 Planung	3
3. Projektdurchführung	4
3.1 Vorbereitung	4
3.2 Entwicklung von der Login Maske	4
3.3 Entwicklung zum Anlegen eines neuen Schadensfalls	4
3.4 Entwicklung des LBM internen-Katalogs	5
3.5 Senden an Schadensteam eines erfassten Schadensfalls	5
3.6 Abschließen eines Schadensfalls	5
3.7 Technische Umsetzung im Detail	6
3.7.1 Architektur und Schichtenmodell	6
3.7.2 Authentifizierung und Berechtigungssteuerung	6
3.7.3 Workflow und Statuslogik der Schadensfälle	6
3.7.4 Formular- und Validierungslogik	6
3.7.5 Katalogfunktionen und Kostenberechnung	6
3.7.6 Administrative Funktionen	7
3.7.7 Technische Grenzen im Projektzeitraum	7
4. Testphase	7
4.1 Testen der Login Seite mit Rechteprüfung	7
4.2 Testen der Funktion zum Anlegen eines neuen Schadensfalls	7
4.3 Testen der Funktion des LBM internen-Katalogs	8
4.4 Testen der Funktion zum Senden eines Schadenfalls	8
4.5 Testen der Funktion zum Abschließen eins Schadenfalls	8
4.6 Erweiterte Tests und Qualitätsbetrachtung	8
4.6.1 Testvorgehen über den Gesamtprozess	8
4.6.2 Qualitätssicherung der Daten	8
4.6.3 Betriebsaspekte und Wartbarkeit	8

4.6.4 Eingesetzte Testmodelle	9
5. Projektabschluss	9
5.1 Innerbetriebliche Dokumentationen.....	9
5.2 Soll-Ist-Vergleich.....	9
5.3 Fazit.....	10
5.4 Ausblick	10

Vorwort

Syntaktische Hinweise

Fachwörter werden einmalig fett und kursiv hervorgehoben und im Glossar erläutert.

Verweise auf den Anhang werden kursiv dargestellt und enthalten jeweils Anlagennummer und Seitenzahl nach dem folgenden Schema: (*siehe Abbildung XX – Seite A-XX*).

Anhang

Der Anhang wird durch ein eigenes Inhaltsverzeichnis gegliedert.

Quellen

Die verwendeten Quellen werden am Ende der Dokumentation als Teil des Anhangs angegeben.

Datenschutz

Aufgrund betriebsinterner Datenschutz- und Informationssicherheitsrichtlinien können in der Dokumentation und deren Anhängen keine personenbezogenen und betriebsinternen Daten gezeigt werden.

1. Projektbeschreibung

1.1 Einleitung

Diese Dokumentation wurde im Rahmen der betrieblichen Projektarbeit zur Berufsausbildung zum Fachinformatiker für Anwendungsentwicklung erstellt. Das Projekt umfasst die Neuentwicklung der bestehenden Software **SaS2** (siehe Abbildung 2 – Seite A-8).

1.2 Projektumfeld

Der **LBM** ist eine Behörde, die dem **MWVLW** untergeordnet ist. Der LBM ist verantwortlich für die Planung, Instandhaltung und Neubau von Straßen, und Radwegen die nicht dem Bund gehören. Außerdem ist der LBM als Luftfahrtbehörde für die Überwachung und Genehmigung im Bereich des Luftverkehrs zuständig.

Der Landesbetrieb Mobilität Rheinland-Pfalz beschäftigt insgesamt 3200 Mitarbeiter. Der Hauptsitz befindet sich in Koblenz, wo ca. 450 Mitarbeiter tätig sind. Die restlichen Mitarbeiter sind auf die acht regionalen Dienststellen verteilt, denen wiederum, Straßenmeistereien und weitere Liegenschaften nachgeordnet sind (siehe Quellenverzeichnis – Seite A-3).

Die Fachgruppe Informationstechnologie befindet sich am Hauptstandort Koblenz und besteht aus ca. 60 Mitarbeitern. Sie ist in die Fachteams IT-Service und IT-Organisation, IT-Anwendungen und Verfahren, IT-Hardware und Infrastruktur sowie SAP-Systeme unterteilt (siehe Abbildung 1 – Seite A-4).²

(² siehe Quellenverzeichnis – Nr. 2)

1.3 Projektziel

Das Ziel des Projektes ist die Neuentwicklung zentraler Bausteine der Software für eine spätere komplette Eigenentwicklung. Im Projekt wurden der Anmeldescreen mit **Rechteprüfung**, die Eingabemaske zur Fallanlage, der interne Katalog sowie der **Workflow** zur Weiterleitung und Bearbeitung von Schadensfällen umgesetzt.

1.4 Projektzeitraum

Das Projekt wird im Zeitraum vom 01.03.2026 bis zum 23.04.2026 durchgeführt.

1.5 Projektbetreuung und Entwicklungsumgebung

Das Projekt wird von Herrn Christian Willms für fachliche Fragen und Armin Schäfer für technische Fragen betreut und überwacht (siehe Tabelle 1: Seite A-4). Entwickelt wird auf einem **WSL-Ubuntu** 24.04 LTS, welches auf einem Dell Precision 7790 mit einem Intel Core i9 und 64 GB Arbeitsspeicher läuft. Die **Entwicklungsumgebung** ist Visual Studio Code und der Hauptsprache JavaScript.

1.6 Anforderungsanalyse mit Auftraggeber

Es sollen folgenden Anforderungen vom Auftraggeber in der Neuentwicklung der Software SaS2 berücksichtigt werden:

- **Webbasierte Anwendung:** Die Software soll als Webanwendung im LBM-Netz bereitgestellt werden.
- **Benutzeroberfläche nach dem Corporate Design des LBM:** Die Benutzeroberfläche der Webanwendung soll dem **Corporate Design** des LBMs entsprechen.
- **Authentifizierung der Benutzer durch das Microsoft Active-Directory:** Die Mitarbeiter sollen sich über das Microsoft **Active-Directory authentifizieren** können.
- **Gruppen beschränkte Rechte:** Anhand von **Gruppenmitgliedschaften** gibt es verschiedene Rechte die für bestimmte Tätigkeiten erforderlich sind.
- **Dateiupload für Beweisdaten:** Es soll die Möglichkeit bestehen, Bilder oder andere Beweise in der Software hochzuladen für spätere Rückfragen seitens der Versicherung oder anderer Stellen. Diese müssen Rechtssicher abgespeichert werden.
- **Schnittstelle zu SAP:** Es soll die Möglichkeit geben, nach dem Versand der Rechnung den Fall in SAP zu übertragen, damit der Zahlungseingang gebucht werden kann. Nach dem der Zahlungseingang verbucht wurde, soll automatisch der Fall in **SaS3** als geschlossen markiert werden.

Diese Anforderungen sind elementarer Bestandteil für die weitere Planung und Entwicklung der Webanwendung SaS3.

1.7 Änderung gegenüber dem Projektantrag

Im Projekt wurden im Vergleich zum Antrag keine inhaltlichen Änderungen vorgenommen (siehe Tabelle 3 – Seite A-4).

2. Projektplanung

2.1 Ist-Analyse

Die Software Sas2 dient der Erfassung aller Schäden an Straßeneinrichtungen (siehe Abbildung 3 bis 8 – Seite A-8 bis A-11). Dort werden alle Schäden, die an jeglichen Straßeneinrichtungen erfolgen, dokumentiert. Auch werden durch diese Software die jeweiligen Rechnungen und Bescheide an die Verursacher versendet. Nach Beendigung der Erfassung und nach dem versenden der Rechnung, wird der Schadensfall an SAP weitergeleitet, damit der Zahlungseingang korrekt gebucht werden kann. Falls notwendig, werden in der Software Beweise gesammelt und gespeichert, um diese für spätere Rückfragen schon bereit liegen zu haben. Es gibt insgesamt drei wichtige Bereiche innerhalb der Software. Dies ist die Erfassung des Schadens, der von den einzelnen Straßenmeistereien gemacht wird. Nachdem die Schadenserfassung vollständig abgeschlossen wurde, wird der Fall an das Team der Schadensabteilung weitergeleitet, welche dann die Rechnungen schreiben. Beim Erstellen der Rechnungen gilt das **Vier-Augen-Prinzip**, so dass die Fachgruppenleitung den Fall als korrekt bestätigen muss. Wenn die Rechnungen geschrieben wurden, wird der Fall an das Finanzteam weitergeleitet, für die Zuordnung der Beträge. Die Software ist allerdings schon Anfang der 2000er entwickelt worden. Dadurch entspricht diese nicht mehr dem aktuellen Stand. Ebenso ist die Performance stark eingeschränkt, da die Software nicht mehr in einer aktuellen Programmiersprache entwickelt wurde.

2.2 Soll-Konzept

Es soll eine komplette, webbasierte Neuentwicklung entstehen, die den Workflow verbessert und an das Corporate Design des LBM angepasst ist. Außerdem soll die Anwendung den neuen internen Richtlinien des LBM entsprechen. Durch eine moderne Webanwendung kann auch die Geschwindigkeit verbessert werden. Die Wartbarkeit des Systems wurde deutlich vereinfacht und ist somit besser, was den administrativen Aufwand erheblich reduziert..

Folgende Anforderungen sollen gemäß der 80 Stunden, die für das Projekt bereitstehen, umgesetzt werden:

1. Eine Login-Seite die mit dem AD des LBM kommuniziert und die Anmeldung mit dem normalen Benutzer ermöglicht.
2. Die Rechte sind von den AD-Gruppenmitgliedschaften abhängig und werden auch entsprechend zugewiesen.
3. Das Design entspricht dem Corporate Design des LBMs für ein einheitliches Auftreten.
4. Es kann vom Erfasser ein neuer Schadensfall angelegt werden und alle notwendigen Werte können eingetragen werden.
5. Der Erfasser kann den Schadensfall nach Abschluss an das Schadensteam für die abschließende Bearbeitung weiterleiten.
6. Das Schadensteam hat Zugriff auf alle Fälle, kann diese sichten, bearbeiten und letztlich abschließen.

2.3 Planung

Aus organisatorischer Sicht hat sich gezeigt, dass ein klarer Kommunikationsrhythmus zwischen Entwicklung und Fachbetreuung wesentlich für die Projektsicherheit ist. Rückmeldungen zu fachlichen Detailfragen wurden dadurch nicht gesammelt, sondern zeitnah verarbeitet. Somit konnten Interpretationsspielräume in der Umsetzung reduziert werden und die Qualität des Ergebnisses war besser. Für zukünftige Ausbaustufen wird empfohlen, dieses Vorgehen beizubehalten und zusätzlich feste Review-Termine für offene **Integrationsanforderungen** wie Dateiapload und SAP-Übergabe einzuplanen.

Risikoseitig wurden insbesondere drei Punkte betrachtet: erstens die technische Abhängigkeit von Active-Directory und Gruppenauflösung, zweitens die **Datenkonsistenz** bei komplexen Statuswechseln und drittens die Qualität externer **Referenzabfragen**. Für jedes Risiko wurden Gegenmaßnahmen festgelegt, zum Beispiel zusätzliche Validierungen, reproduzierbare Testfälle und die Nutzung von **Dummy-Benutzern** für Entkopplungstests. Diese Maßnahmen konnten das Projektrisiko deutlich reduzieren. Gleichzeitig wurde transparent dokumentiert, welche Restaufgaben nicht mehr innerhalb der verfügbaren Projektzeit abgeschlossen werden konnten.

Für die Steuerung des Projektes wurde mit einer Priorisierung nach Muss-, Soll- und Kann-Anforderungen gearbeitet. Muss-Anforderungen waren alle Funktionen, die den Kernprozess der Schadenserfassung direkt betreffen, insbesondere Login, Rechteprüfung, Fallanlage, Teamworkflow und Abschluss. Soll-Anforderungen betrafen Komfort- und Integrationsfunktionen, die fachlich wichtig sind, aber nicht zwingend für einen lauffähigen Kern notwendig waren. Diese Priorisierung war die Grundlage für mehrere Umplanungen innerhalb des Zeitrahmens und hat verhindert, dass der Fokus durch zu viele parallele Themen verloren ging.

Die Projektorganisation wurde so aufgebaut, dass fachliche und technische Entscheidungen früh abgestimmt werden konnten. Fachliche Rückfragen wurden direkt mit der Projektbetreuung geklärt, während technische Umsetzungen in kurzen Entwicklungszyklen umgesetzt und anschließend überprüft wurden. Dieses Vorgehen war für den engen Zeitrahmen entscheidend, weil dadurch Fehlentwicklungen früh erkannt wurden. Statt erst am Projektende größere

Korrekturen durchzuführen, konnten Anpassungen laufend vorgenommen werden. Dadurch blieb der Projektfortschritt trotz technischer Herausforderungen insgesamt stabil und planbar.

3. Projektdurchführung

3.1 Vorbereitung

Zur Vorbereitung wurde auf dem **PostgreSQL-Server Cluster** eine neue Datenbank erstellt, die alle wichtigen Tabellen enthält. Es wurde sich aufgrund von teuren Lizenzmodellen von Microsoft SQL-Server und der grundsätzlichen Ausrichtung des LBMs für die **OpenSource** Variante PostgreSQL entschieden. Die Tabellenstruktur wurden zum Teil aus der alten Software SaS2 übernommen und ein Teil wurde überarbeitet. Beim Erstellen und Überarbeiten der Tabellen, wurden doppelt vorkommende Tabellen im LBM wegelassen und auf allgemein im LBM verfügbare Tabellen gesetzt. Dies vereinfachten die Wartbarkeit und die Aktualisierung der Datensätze erheblich. Diese PostgreSQL-Datenbank dient in erster Linie für die Testzwecke (siehe *ER-Diagramm – Seite A-19*). Für die spätere Produktivumgebung wird es eine neue saubere PostgreSQL-Datenbank geben. Wenn im späteren Verlauf von der Datenbank die Rede ist, ist damit diese Dummy-Datenbank gemeint.

3.2 Entwicklung von der Login Maske

Im ersten Schritt wurde eine statische Login-Seite erstellt, die dem Corporate Design des LBM entspricht. Dabei wurde zunächst nicht die Logik implementiert für die Anmeldung und Rechteprüfung. (siehe *Abbildung 9 – Seite A-11*)

Nach dem Erstellen der statischen Login-Seite, wurde die Logik für die Rechte-Prüfung implementiert und an unser bestehendes Active-Directory angebunden. Dies dient der einfacheren Rechteverwaltung in der Anwendung. Außerdem wurde die Möglichkeit geschaffen sich mit Dummy-Daten anzumelden. Im späteren Verlauf wurde die Möglichkeit, sich mit Dummy-Daten anzumelden Passwort geschützt, damit unbefugte Personen keinen Zugriff auf die Webanwendung haben. Beim Login wird geprüft, ob der Benutzer im AD existiert, ein gültiges Kennwort hat, der letzte Login nicht älter wie sechs Monate alt ist, und in der Gruppe „SaS3-Alle“ ist. Jeder der in der Gruppe „SaS3-Alle“ ist erhält die Berechtigung die Anwendung zu öffnen, jedoch können nur Gruppenmitglieder der Gruppe „SaS3-Admin“ Administrationsaufgaben durchführen. Wenn man in der Gruppe „SaS3-Bearbeiter“ ist kann man auch die Rechnungen erstellen und einen Schadensfall als abgeschlossen kennzeichnen.

Nach dem Login sieht jeder Erfasser eine Tabelle mit allen Schadensfällen, die noch nicht abgeschlossen, oder die noch in seiner Verantwortung liegen (siehe *Abbildung 10 – Seite A-12*).

3.3 Entwicklung zum Anlegen eines neuen Schadensfalls

Nach der Login-Seite mit entsprechender Berechtigungsprüfung, wurde die Eingabemaske zur Erstellung eines neuen Schadensfalls angelegt. Wenn ein neuer Schadensfall erstellt wird, wird direkt eine eindeutige Schadensfallnummer erstellt die dem Format „YYXXXX“ entspricht, wobei YY für das Jahreszahl und XXXX für eine fortlaufende Zahl steht. Diese eindeutige ID ist für alle späteren Funktionen von großer Bedeutung und alles bezieht sich auf diese Nummer. Es wurden die Felder aus der alten Software SaS2 übernommen und in dem Corporate

Design des LBM umgesetzt. Beim Speichern des Schadensfalls werden die Daten in die SaS3 Datenbank geschrieben (siehe Abbildung 11 bis 15 – Seite A-12 bis A-14).

3.4 Entwicklung des LBM internen-Katalogs

Nun wurde der **LBM-interne-Katalog** erstellt. Dieser beinhaltet einmal alle Möglichkeiten die einem Verursacher in Rechnung gestellt werden können. Darunter fallen die Stundensätze der Mitarbeiter-, Materialkosten für neue **Straßeneinrichtungen** und alle weiteren Kosten die in irgendeiner Weise in Rechnung gestellt werden müssen (siehe Abbildung 15 – Seite A-14). Dieser Katalog muss immer aktualisiert und erweitert werden, damit die Kosten in korrekter Weise berechnet werden. Die Aktualisierung des Katalogs obliegt dem Schadensteam. Diese haben durch die Rechteverwaltung auch die Möglichkeit eben solche Tätigkeiten auszuführen (siehe Abbildung 18 bis 20 – Seite A-16 bis A-17).

3.5 Senden an Schadensteam eines erfassten Schadensfalls

Als Abschluss wurde die Funktion implementiert, einen Schadensfall als Erfasser an die nächste Stelle, das Schadensteam, zu senden. Das Schadensteam bekommt alle Fälle in einer übersichtlichen Tabelle angezeigt und muss diese weiterbearbeiten. Danach kann der Erfasser diesen Fall nicht mehr sehen oder bearbeiten. Nur noch das Schadensteam kann diese Fälle bearbeiten und abschließen. Der Fall wird nicht mehr einer Person zugeordnet, sondern dem neuen Team. Sehen können ihn alle Mitglieder, die in der Gruppe „SaS3-Bearbeiter“ sind.

3.6 Abschließen eines Schadensfalls

Das Schadensteam kann sich in einer Tabelle alle Schadensfälle anschauen und abschließend bearbeiten. Alle Schadensfälle, die dem Schadensteam zugewiesen sind werden in einer gesonderten Tabelle, der Übersichtlichkeit halber, dargestellt. Je nach Region des Schadensfalls, werden diese dem für die jeweilige Region zuständigen Sachbearbeiter zugewiesen. Jeder Sachbearbeiter kann sehen, wem welcher Fall zugewiesen ist, oder wer an welchem Fall zurzeit am Arbeiten ist. Der zuständige Sachbearbeiter muss seine zugehörigen Fälle durchschauen. Im Anschluss daran stellt er die Rechnung aus und gibt die Rechnung an den Verursacher weiter. Bei komplizierteren Fällen können diese an die Leitung des Schadensteam geschickt werden. Die Leitung kann den Schadensfall genauso bearbeiten und im Anschluss entweder die Rechnung schreiben oder diesen mit Bemerkungen an den vorherigen Sachbearbeiter zurückschicken. Beim Versenden der Rechnung wird der Fall abgeschlossen und die erforderlichen Informationen werden an die Buchhaltung übergeben, damit dort der Zahlungseingang korrekt verbucht werden kann. Nach dem Abschließen eines Schadensfalls ist es nicht mehr möglich diesen zu bearbeiten. Dieser Fall muss jetzt mit allen Beweisen revisionssicher abgespeichert werden. Allerdings ist diese Funktion nicht implementiert worden. Erst nach Ablauf der geltenden gesetzlichen **Aufbewahrungsfristen** kann dieser gelöscht werden.

3.7 Technische Umsetzung im Detail

3.7.1 Architektur und **Schichtenmodell**

Die Anwendung wurde als Weblösung mit klarer Trennung zwischen Benutzeroberfläche, **Serverlogik** und Datenzugriff umgesetzt. Die Oberflächen übernehmen Erfassung und Benutzerführung, während fachlich verbindliche Regeln zentral im Server verarbeitet werden. Der Datenbankzugriff ist gekapselt umgesetzt. Diese Aufteilung verbessert Wartbarkeit und Stabilität, weil Änderungen kontrolliert in den jeweiligen Schichten vorgenommen werden können.

Die Schichtentrennung hat sich in der Projektdurchführung besonders bei der Fehlersuche bewährt. Darstellungsfehler konnten unabhängig von fachlichen Prüfregeleln behandelt werden. Gleichzeitig blieb die zentrale Regelprüfung konsistent, unabhängig davon, über welche Maske ein Vorgang ausgelöst wurde. (siehe UML-Diagramm – Seite A-18)

3.7.2 Authentifizierung und **Berechtigungssteuerung**

Die Anmeldung erfolgt benutzerbasiert mit Passwortprüfung und anschließender rollenbasierter Rechteaflösung. Für AD-Benutzer werden Gruppeninformationen ausgewertet und daraus Berechtigungen abgeleitet. Zusätzlich wurden Dummy-Benutzer für reproduzierbare Tests verwendet. (siehe Abbildung 17 – Seite A-15)

Die Berechtigungslogik schützt nicht nur Oberflächen, sondern auch API-Endpunkte. Damit sind unzulässige Direktzugriffe ausgeschlossen. Zusätzlich wird pro Schadensfall geprüft, ob ein Benutzer den Fall nur sehen oder auch bearbeiten darf.

3.7.3 Workflow und Statuslogik der Schadensfälle

Der Fallworkflow wurde als verbindliche Statuslogik umgesetzt. Ein neuer Fall startet im Status Neu und wird nach Erfüllung definierter Voraussetzungen in die weiteren Bearbeitungsstufen überführt. Übergänge werden serverseitig geprüft und nicht allein durch die Oberfläche gesteuert.

Besonders wichtig ist die Regel Kosten komplett erfasst. Dieser Status darf nur gesetzt werden, wenn Pflichtinformationen vollständig vorliegen. Danach werden kostenrelevante Felder gesperrt, um die Nachvollziehbarkeit der Abrechnung sicherzustellen.

3.7.4 Formular- und Validierungslogik

Die Erfassungsmaske nutzt mehrstufige Validierungen. Bereits bei der Eingabe werden Formate und Pflichtangaben geprüft. Zusätzlich erfolgen serverseitige Prüfungen vor Speicherung und Statuswechsel. Dadurch bleibt die Datenqualität konsistent.

Auch für Ausnahmesituationen wurde ein robustes Verhalten umgesetzt. Bei temporär nicht verfügbaren Referenzabfragen werden verständliche Hinweise ausgegeben, ohne dass der gesamte Arbeitsablauf unkontrolliert abbricht.

3.7.5 Katalogfunktionen und Kostenberechnung

Der Kostenprozess nutzt den LBM-Katalog sowie Versicherungs- und Kassenkatalog. Positionen werden mit Menge und Satz einem Fall zugeordnet und daraus nachvollziehbar summiert. Zusätzlich werden Versicherungs- und Kasseninformationen strukturiert in die Bearbeitung übernommen.

Die Katalogpflege erfolgt über administrative Funktionen mit Berechtigungsprüfung. Einträge können angelegt, geändert und deaktiviert werden, ohne dass Quellcode angepasst werden muss. Das reduziert den Wartungsaufwand und erhöht die fachliche Flexibilität. Das Schadensteam darf alle Kataloge bearbeiten (siehe Abbildung 18 bis 20 – Seite A-16 bis A-17).

3.7.6 Administrative Funktionen

Die zusätzlichen Funktionen für den Administrator werden neben den normalen Funktionen für den Schadensteammitarbeiter angezeigt. (siehe Abbildung 16 – Seite A-15)

Der Administrator kann zusätzlich noch die Rechtevergabe ändern (siehe Abbildung 17 – Seite A-15) und eine Datenbank verbinden (siehe Abbildung 21 – Seite A-17).

3.7.7 Technische Grenzen im Projektzeitraum

Dateiupload und **SAP-Schnittstelle** wurden im Projektzeitraum nicht umgesetzt. Beide Themen erfordern zusätzliche technische und organisatorische Bausteine, insbesondere für sichere Ablageprozesse und belastbare Schnittstellenkommunikation mit Rückmeldung. Sie wurden daher transparent in eine nächste Ausbaustufe verschoben.

4. Testphase

4.1 Testen der Login Seite mit Rechteprüfung

Als erster Test wurde die Login Seite mit anschließender Rechteprüfung getestet. Dies geschah durch **Dummy-Benutzer**, die in verschiedenen Gruppen waren, wodurch jeder die entsprechenden Berechtigungen bekommen sollte. Das Ergebnis war genauso wie erwartet, in dem die jeweiligen Benutzer auch die korrekten Rollen zugewiesen bekommen haben.

Beim Testen der Anmeldung mit AD-Benutzern, kam es häufig zu Fehlern mit Zugriffsproblemen. Grund dafür war eine fehlerhafte Konfiguration der AD-Gruppen, die auf die Anwendung zugreifen dürfen. Nachdem die Gruppen korrekt in der Software zugewiesen wurden, gab es allerdings Schwierigkeiten mit den Rechten, da die Gruppen nicht immer vollständig aus dem AD ausgelesen wurden. Es lag daran, dass der Gruppenname, wenn er länger wie 20 Zeichen war, abgeschnitten wurde. Das konnte behoben werden, indem nicht der **sAMAccountName** ausgelesen wird, sondern der **CommonName**. Danach war es möglich auf die Anwendung mit AD-Benutzern zuzugreifen. Ebenso wurde die Konfiguration der Datenbankverbindung korrigiert, und eine Verbindung war möglich. Hier lag der Fehler darin, dass ein falscher Benutzer für die Anmeldung an der Datenbank übergeben wurde. (siehe Abbildung 22 – Seite A-18). (siehe Prüfprotokoll – Seite A-36)

4.2 Testen der Funktion zum Anlegen eines neuen Schadensfalls

Die Eingabemaske wurde auf alle wichtigen Bestandteile geprüft. Das Testen beinhaltet folgende Aspekte:

1. Automatische Generierung von Schadensfallnummer: Die eindeutige Nummer des Schadensfalls soll automatisch generiert werden und nicht veränderbar sein.
2. Korrekte Netzknoten: Der Schadensabschnitt wurde korrekt aus anderen Datenbanken eingelesen
3. **KBA-Prüfung**: Das Verursacher KFZ-Kennzeichen kann einem Kreis oder einer Stadt eindeutig zugeordnet werden.
4. Verursacher-Adresse: Der Verursacher hat eine gültige Adresse.

Das Ergebnis des ersten Tests war nicht wie erwartet, da dort die KBA-Prüfung nicht korrekt in der Datenbank abgefragt wurde. Dadurch konnte nicht eindeutig zugeordnet werden.

Beim zweiten Test war das Ergebnis dann wie erwartet und die KBA-Prüfung war nun eindeutig durchführbar. (siehe Prüfprotokoll – Seite A-36)

4.3 Testen der Funktion des LBM internen-Katalogs

Es wurde geprüft ob beim Anlegen eines neuen Schadenfalls der LBM-interne-Katalog korrekt geladen wird und beim Auswählen einer Position auch alle Kosten korrekt berechnet werden. Das Ergebnis war wie erwartet, außer dem Berechnen der Kosten. Dort wurde die Logik angepasst und nochmals getestet. Danach war das Ergebnis korrekt.

4.4 Testen der Funktion zum Senden eines Schadenfalls

Es wurde sich als Schadenserfasser angemeldet und ein neuer Schadensfall angelegt. Nach Abschluss des Anlegens wurde dieser an das Schadensteam weitergeleitet. Im Anschluss wurde sich als Schadensteam-Mitarbeiter angemeldet und geprüft, ob dieser dann auch dort angezeigt wird.

Das Ergebnis war wie erwartet und der Schadensfall wurde dem Mitarbeiter des Schadensteams korrekt angezeigt.

4.5 Testen der Funktion zum Abschließen eines Schadenfalls

Es wurde sich als Schadensteammitarbeiter mit Dummy-Daten angemeldet. Im Anschluss wurde ein offener Schadensfall ausgesucht und dieser wurde mit allen noch notwendigen Informationen befüllt. Als Abschluss wurde die Rechnung generiert und der Fall als geschlossen markiert.

4.6 Erweiterte Tests und Qualitätsbetrachtung

4.6.1 Testvorgehen über den Gesamtprozess

Die Tests wurden prozessorientiert entlang des gesamten Ablaufs durchgeführt: Anmeldung, Fallanlage, Weiterleitung, Team-Bearbeitung und Abschluss. Zusätzlich wurden Wiederholungstests nach Korrekturen ausgeführt, um die Stabilität der Anpassungen zu prüfen.

Neben **Positivtests** wurden Fehlerszenarien geprüft, etwa unvollständige Eingaben, fehlende Berechtigungen und unzulässige Statuswechsel. Dadurch wurde sichergestellt, dass die Anwendung in Fach- und Fehlerfällen nachvollziehbar reagiert.

4.6.2 Qualitätssicherung der Daten

Ein Schwerpunkt lag auf der Konsistenz gespeicherter Daten. Pflichtfelder, **Feldabhängigkeiten** und Statusvoraussetzungen wurden systematisch getestet. Besonders relevant war, dass die **Sperrlogik** kostenrelevanter Felder nach dem Setzen des Status „Kosten komplett erfasst“ in korrekter Form funktioniert und alle betreffenden Eingabefelder gesperrt sind.

Für reproduzierbare Ergebnisse wurden strukturierte Dummy-Daten verwendet. Das erleichterte die Fehlersuche und ermöglichte vergleichbare **Regressionstests**.

4.6.3 Betriebsaspekte und Wartbarkeit

Neben Fachfunktionen wurden Betriebsaspekte bewertet, darunter Fehlermeldungen, Protokollierung, Konfigurierbarkeit und zentral gepflegte Geschäftsregeln. Diese Punkte sind wesentlich für einen stabilen Regelbetrieb.

Insgesamt zeigt die erweiterte Betrachtung, dass der umgesetzte Kern fachlich nutzbar und technisch ausbaufähig ist. Offene Anforderungen sind klar dokumentiert und für die nächste Ausbaustufe priorisierbar.

4.6.4 Eingesetzte Testmodelle

Es wurden ausschließlich **White-Box-Tests** durchgeführt, bei dem der Entwickler weiß, was als Ergebnis rauskommen muss. Diese Testmethode ist für den Anfang der Entwicklung einfacher und nicht so zeitintensiv, da man mit konkreten Werten die Anwendung testen kann. Als Abschluss und vor der Übergabe an den Kunden sollten **Black-Box-Tests** gemacht werden, damit sichergestellt werden kann, dass die Anwendung auch in Verwendung eines nicht Entwicklers, die Funktionen ordnungsgemäß funktionieren.

5. Projektabschluss

5.1 Innerbetriebliche Dokumentationen

Die Entwicklerdokumentation ist im Anhang hinterlegt (*siehe Anhang – Seite A-26*). Ebenso ist die Dokumentation zur Bedienung der Software für einen Anwender im Anhang hinterlegt (*siehe Anhang – Seite A-20*). Da im LBM alle Dokumentationen nur noch im eigenen Wiki-System gehalten werden, sind diese als Markdown formatiert und weisen nicht die Standardformatierung von Textdokumenten auf.

5.2 Soll-Ist-Vergleich

Der Soll-Ist-Vergleich wurde anhand der implementierten Funktionen im Quellcode durchgeführt (server.mjs, src/create-damage-case.js und zugehörige Oberflächen).

Umgesetzt:	Login mit Active-Directory-Anbindung und Passwortprüfung.
Umgesetzt:	Rollen- und Rechtekonzept über Gruppenmitgliedschaften.
Umgesetzt:	Oberfläche im Corporate Design des LBM.
Umgesetzt:	Anlegen, Bearbeiten, Weiterleiten und Abschließen von Schadensfällen.
Umgesetzt:	LBM-interner Katalog sowie Versicherungs- und Kassenkatalog.
Nicht umgesetzt:	Dateiupload für Beweisdaten.
Begründung:	Im aktuellen Projektstand sind keine Upload-Endpunkte und keine persistente Dateiablage implementiert. Die verfügbare Projektzeit wurde auf Authentifizierung, Rechteprüfung und den Kernworkflow der Fallbearbeitung priorisiert.
Nicht umgesetzt:	Schnittstelle zu SAP für Rechnungsübergabe und Zahlungseingang.
Begründung:	Im Quellcode ist keine technische SAP-Integration vorhanden (keine SAP-API-Anbindung, kein Übergabeprozess, keine Rückmeldung zum Zahlungseingang). Diese Anforderung wurde in eine spätere Ausbaustufe verschoben.

(siehe Tabelle 3 – Seite A-6)

5.3 Fazit

Bei der Umsetzung traten vor allem bei der AD-Authentifizierung und der stabilen Rollenauflösung zeitkritische Herausforderungen auf. Dadurch mussten Entwicklungsressourcen priorisiert auf den Kernprozess gelegt werden. Aus diesem Grund wurden Dateiupload und SAP-Schnittstelle nicht innerhalb des Projektzeitraums umgesetzt.

Positiv war die **iterative Vorgehensweise**: Ein lauffähiger Kern konnte früh getestet und in mehreren Schritten stabilisiert werden.

Insgesamt wurde eine tragfähige Grundlage für die weitere Entwicklung geschaffen. Besonders erfolgreich waren Login, Rechtekonzept, Fallworkflow und Katalogfunktionen.

(siehe Abbildung 23 – A-41)

5.4 Ausblick

In der nächsten Ausbaustufe werden Dateiupload mit **revisionssicherer Ablage** sowie die SAP-Schnittstelle zur Übergabe und Rückmeldung von Zahlungsinformationen umgesetzt.

Die Umsetzung erfolgt stufenweise: zuerst Dateiupload mit Prüf- und Ablagekonzept, danach die SAP-Anbindung mit definiertem Übergabeformat und Rückkanal.

Für beide Stufen werden vorab klare Abnahmekriterien festgelegt, zum Beispiel Dateiformate, Reaktionszeiten, Fehlerklassen und verbindliche Statusregeln.

Durch diesen Fahrplan bleibt der aktuelle Kernprozess stabil, während offene Anforderungen kontrolliert und nachvollziehbar ergänzt werden.

Für die nächste Umsetzungsphase wird ein verbindliches **Meilenstein-Reporting** empfohlen, um die offenen Anforderungen Dateiupload und SAP-Schnittstelle planbar zu realisieren. Jeder Meilenstein soll einen klaren fachlichen Scope, eine technische **Definition of Done** sowie ein dokumentiertes Testergebnis enthalten. Zusätzlich sollten Abhängigkeiten zwischen Fachbereich, Entwicklung und Betrieb transparent aufgeführt werden. Durch diese Struktur kann früh erkannt werden, ob sich kritische Pfade verschieben, und es können rechtzeitig Gegenmaßnahmen eingeleitet werden. Das reduziert die Gefahr, dass Integrationsaufgaben sich erneut in den Projektabschluss verschieben.

Ergänzend ist eine regelmäßige Fortschrittsbewertung sinnvoll, bei der Aufwand, Risiken und Qualitätsstand je Teilfunktion bewertet werden. Dabei sollte nicht nur der Entwicklungsstand betrachtet werden, sondern auch die Betriebsreife, zum Beispiel Protokollierung, Fehlerbehandlung, Berechtigungskonzept und Abnahmefähigkeit. Ein einheitliches Bewertungsraster unterstützt die Vergleichbarkeit über mehrere Iterationen hinweg und erleichtert die Priorisierung bei begrenzten Ressourcen. Auf diese Weise bleibt der bereits erreichte Kernprozess stabil, während Erweiterungen kontrolliert und nachvollziehbar ergänzt werden.

Anhang

Anhangsverzeichnis

Glossar	1
Quellenverzeichnis	3
Anlagen	4
Organisationsplan der Fachgruppe Informations-Technologie	4
Projektschnittstellen	4
Zeitplanung	5
Soll-Ist-Vergleich	6
SaS2 – Login Maske der alten Software	7
SaS2 – Übersicht aller eigenen Schadensfälle	7
SaS2 – Schadensfall anlegen - Schaden	8
SaS2 – Schadensfall anlegen - Wo/Wer	8
SaS2 – Schadensfall anlegen - Haftung	9
SaS2 – Schadensfall anlegen - Kosten	9
SaS2 – Schadensfall anlegen - Übersicht	10
SaS3 – Login Maske	10
SaS3 – Übersicht als Erfasser	11
SaS3 – Schadensfall anlegen – Allgemeine Daten	11
SaS3 – Schadensfall anlegen – Unfallort	12
SaS3 – Schadensfall anlegen – Versicherungsauswahl	12
SaS3 – Schadensfall anlegen – Auswahl Haftung	13
SaS3 – Schadensfall anlegen – Auswahl Kosten	13
SaS3 – Administrator – Übersicht mit Notwendigen Funktionen	14
SaS3 – Administrator – Rechteverwaltung	14
SaS3 – Administrator – Kassenkatalog bearbeiten	15
SaS3 – Administrator – Kostenkatalog bearbeiten	15
SaS3 – Administrator – Versicherungskatalog bearbeiten	16
SaS3 – Administrator – Datenbankverbindung konfigurieren	16
SaS3 – Übersicht mit Datenbankverbindungsfehler	17
SaS3 – UML-Diagramm	17

SaS3 – ER-Diagramm.....	18
Anwender Dokumentation.....	19
Administrator Dokumentation.....	25
SaS3 – Prüfprotokoll	36
Genehmigter Projektantrag	40

Abbildungsverzeichnis

Abbildung 1: Organisationsplan der Fachgruppe Informations-Technologie	4
Abbildung 2: SaS2 - Login Maske der alten Software	7
Abbildung 3: SaS2 – Übersicht aller eigenen Schadensfälle.....	7
Abbildung 4: SaS2 - Schadensfall anlegen – Schaden	8
Abbildung 5: SaS2 – Schadensfall anlegen - Wo/Wer	8
Abbildung 6: SaS2 – Schadensfall anlegen - Haftung	9
Abbildung 7: SaS2 – Schadensfall anlegen - Kosten	9
Abbildung 8: SaS2 – Schadensfall anlegen - Übersicht	10
Abbildung 9: SaS3 – Login Maske	10
Abbildung 10: SaS3 – Übersicht als Erfasser	11
Abbildung 11: SaS3 – Schadensfall anlegen – Allgemeine Daten.....	11
Abbildung 12: SaS3 – Schadensfall anlegen – Unfallort	12
Abbildung 13: SaS3 – Schadensfall anlegen – Versicherungsauswahl.....	12
Abbildung 14: SaS3 – Schadensfall anlegen – Auswahl Haftung.....	13
Abbildung 15: SaS3 – Schadensfall anlegen – Auswahl Kosten	13
Abbildung 16: SaS3 – Administrator – Übersicht mit Notwendigen Funktionen.....	14
Abbildung 17: SaS3 – Administrator – Rechteverwaltung	14
Abbildung 18: SaS3 – Administrator – Kassenkatalog bearbeiten.....	15
Abbildung 19: SaS3 – Administrator – Kostenkatalog bearbeiten	15
Abbildung 20: SaS3 – Administrator – Versicherungskatalog bearbeiten.....	16
Abbildung 21: SaS3 – Administrator – Datenbankverbindung konfigurieren.....	16
Abbildung 22: SaS3 – Übersicht mit Datenbankverbindungsfehler	17
Abbildung 23: Genehmigter Projektantrag	44

Tabellenverzeichnis

Tabelle 1: Projektschnittstellen	4
Tabelle 2: Projektschnittstellen	5
Tabelle 3: Soll-Ist-Vergleichs	7

Glossar

Begriff	Erklärung	Kontext in der Dokumentation
SaS2	Bestehende Altanwendung zur Schadenserfassung, die abgelöst werden soll	Projektbeschreibung, Ist-Analyse
SaS3	Neue Webanwendung als Nachfolger für SaS2	Projektziel, Soll-Konzept
LBM	Landesbetrieb Mobilität Rheinland-Pfalz	Projektumfeld
MWVLW	Ministerium für Wirtschaft, Verkehr, Landwirtschaft und Weinbau, nächstes Ministerium über dem LBM ³	Projektumfeld
WSL-Ubuntu	Linux-Entwicklungsumgebung unter Windows Subsystem for Linux	Projektbetreuung und Entwicklungsumgebung
Active Directory (AD)	Microsoft-Verzeichnisdienst für Benutzer und Gruppen	Anforderungen, Login, Testphase
Authentifizierung	Prüfung von Benutzeridentität (z. B. Login mit Passwort)	Anforderungen, technische Umsetzung
Berechtigungssteuerung	Vergabe und Prüfung von Zugriffsrechten je Benutzer/Gruppe	Technische Umsetzung 3.7.2
Gruppenmitgliedschaft	Mitgliedschaft in AD-Gruppen als Grundlage für Rollen/Rechte	Anforderungen, Login, Soll-Ist-Vergleich
Rechteprüfung	Fachliche/technische Prüfung, ob eine Aktion erlaubt ist	Login, Testphase, Soll-Ist-Vergleich
Webbasierte Anwendung	Anwendung, die über Browser im Netz genutzt wird	Anforderungen, Soll-Konzept
Corporate Design	Einheitliche visuelle Gestaltungsrichtlinien der Organisation	Anforderungen, Umsetzung
Workflow	Definierter fachlicher Ablauf eines Schadensfalls	Projektziel, technische Umsetzung
Vier-Augen-Prinzip	Freigabeprinzip mit zusätzlicher Prüfung durch zweite Person/Funktion	Ist-Analyse
Dummy-Benutzer	Testkonten für reproduzierbare Rechte- und Funktionstests	Login-Umsetzung, Testphase
Positivtest	Test eines erwarteten gültigen Ablaufs	Erweiterte Testbetrachtung 4.6.1
Regressionstest	Wiederholungstest nach Korrekturen zur Stabilitätssicherung	4.6.2
Datenkonsistenz	Widerspruchsfreie und regelkonforme Datenhaltung	Risikobetrachtung, 4.6.2
Feldabhängigkeit	Regel, bei der die Gültigkeit eines Feldes von anderen Feldern abhängt	4.6.2
Sperrlogik	Mechanismus, der Felder nach bestimmten Statuswerten sperrt	4.6.2
Schichtenmodell	Architekturmuster mit Trennung von UI, Serverlogik und Datenzugriff	3.7.1
Serverlogik	Zentrale fachliche Verarbeitung auf dem Server	3.7.1
Referenzabfrage	Externe oder interne Datenabfrage zur Unterstützung der Eingabe	3.7.4

Revisionssichere Ablage	Manipulationssichere, nachvollziehbare Aufbewahrung von Daten/Dateien	3.6, 5.4	
Aufbewahrungsfrist	Gesetzlich definierter Zeitraum für Datenaufbewahrung	3.6	
SAP-Schnittstelle	Technische Anbindung an SAP für Übergabe und Rückmeldungen	Anforderungen, Ausblick	3.7.6,
Iterative Vorgehensweise	Entwicklung in wiederholten Schritten mit frühem Testen	Fazit 5.3	
Integrationsanforderung	Anforderung mit Anbindung an externe Systeme/Prozesse	Projektplanung, 3.7.6	
Meilenstein-Reporting	Regelmäßige Fortschrittsdokumentation je Projektmeilenstein	5.5	
Definition of Done	Klare Kriterien, wann eine Aufgabe fachlich/technisch abgeschlossen ist	5.5	
PostgreSQL	PostgreSQL ist ein objekt-relationales Open-Source-Datenbankmanagementsystem, welches die Einhaltung von SQL-Standards versucht und eine einfache Erweiterbarkeit sowie Zuverlässigkeit bietet. ⁴	3.1	
Betriebsreife	Eignung einer Funktion für den stabilen produktiven Betrieb	5.5	
Black-Box Test	Der Tester hat keinen Zugriff auf den Quellcode und kann diesen nur durch Eingaben testen.	4.6	
White-Box Test	Der Tester hat vollen Zugriff auf den Quellcode und kann diesen gezielt testen.	4.6	

Quellenverzeichnis

1. Landesbetrieb Mobilität Rheinland-Pfalz Hauptverwaltungsgebäude Koblenz, Luftbild
URL: https://www.webbaviation.de/galerie/_data/i/galleries/Rheinland-Pfalz/Koblenz/Landesbetrieb-Mobilitaet-Rheinland-Pfalz-Hauptverwaltungsgebäude-cb30937-me.jpg
[Stand: 21.04.2026]
2. Landesbetrieb Mobilität Rheinland-Pfalz: „70 Jahre gut unterwegs | Landesbetrieb Mobilität Rheinland-Pfalz (rlp.de)“
URL: <https://lbm.rlp.de/ueber-uns/wir-sind-der-lbm>
[Stand: 21.04.2026]
3. Ministerium für Wirtschaft, Verkehr, Landwirtschaft und Weinbau
URL <https://mwvlw.rlp.de/themen/verkehr>
[Stand 21.04.2026]
4. PostgreSQL: The World's Most Advanced Open Source Relational Database
URL: <https://www.postgresql.org/about/>
[Stand: 21.04.2026]

Anlagen

Organisationsplan der Fachgruppe Informations-Technologie

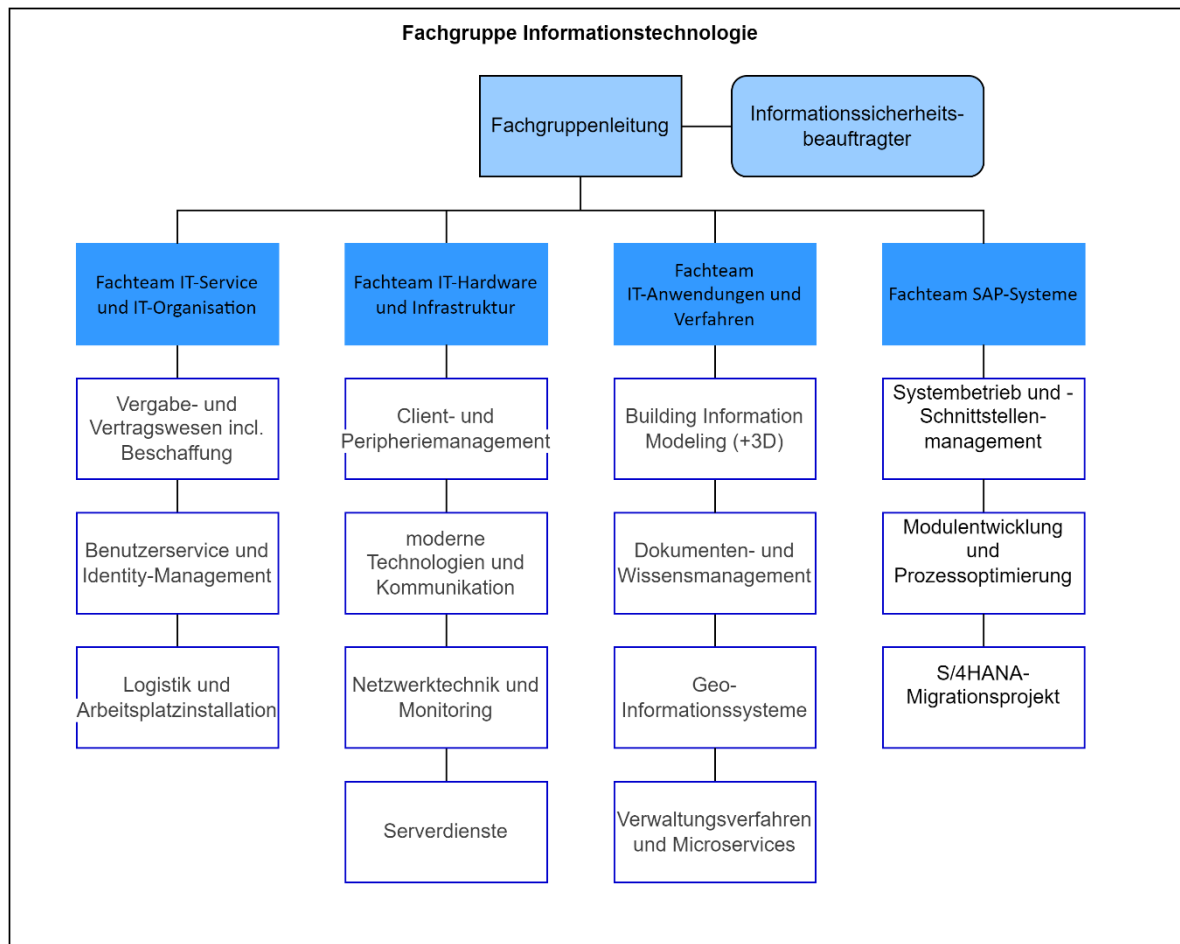


Abbildung 1: Organisationsplan der Fachgruppe Informations-Technologie

Projektschnittstellen

Name	Firma	Abteilung	Beschreibung
Armin Schäfer	LBM RP	IT-Anwendungen und Verfahren	Auftraggeber
Christian Willms	LBM RP	IT-Anwendungen und Verfahren	Projektbetreuer

Tabelle 1: Projektschnittstellen

Zeitplanung

Projektphase	Stunden-Planung
1. Analyse	5h
a. Anforderungsanalyse mit Auftraggeber	2h
b. Ist-Analyse	1h
c. Soll-Konzept erarbeiten	2h
2. Planung	3h
a. Erstellung von UML, Use-Case Diagrammen	2h
b. Rücksprache mit Auftraggeber	1h
3. Realisierung	38h
c. Projekt anlegen mit Projektstruktur	4h
d. Datenbankdesign erstellen	4h
e. Datenbanken befüllen	4h
f. Anmeldescreen erstellen	2h
g. Funktion zur Rechteprüfung erstellen	6h
h. Erstellen der Funktion und Eingabemaske für Straßenmeister:innen	8h
i. Erstellung einer Funktion, für Dateiuploads und Speicherung	3h
j. Erstellen der Funktion und Eingabemaske Schadensteammitarbeiter	7h
4. Testphase	20h
a. Testen des Anmeldescreens und Rechteprüfung	4h
b. Testen der Funktion für Dateiuploads und der Eingabemaske	8h
c. Abschlusstest aller Funktionen	8h
5. Abschluss	14h
a. Dokumentation für den Anwender	4h
b. Dokumentation für den Entwickler	8h
c. Betriebsinterne Dokumentation	2h
Gesamt	80h

Tabelle 2: Projektschnittstellen

Soll-Ist-Vergleich

Projektphase	Soll-Stunden	Ist-Stunden	Begründung
6. Analyse	5h	5h	
d. Anforderungsanalyse mit Auftraggeber	2h	2h	
e. Ist-Analyse	1h	1h	
f. Soll-Konzept erarbeiten	2h	2h	
7. Planung	3h	3h	
k. Erstellung von UML, Use-Case Diagrammen	2h	2h	
l. Rücksprache mit Auftraggeber	1h	1h	
8. Realisierung	38h	39h	
m. Projekt anlegen mit Projektstruktur	4h	2h	Struktur wurde durch IDE automatisch angelegt
n. Datenbankdesign erstellen	4h	6h	Datenbankstruktur unübersichtlicher als gedacht
o. Datenbanken befüllen	4h	4h	
p. Anmeldescreen erstellen	2h	2h	
q. Funktion zur Rechteprüfung erstellen	6h	10h	Verbindung zum AD-Server komplizierter als gedacht
r. Erstellen der Funktion und Eingabemaske für Straßenmeistereien	8h	6h	Die Eingabemaske einfacher als gedacht
s. Erstellung einer Funktion, für Dateiuploads und Speicherung	3h	0h	Durch die Zeitverschiebung der AD-Abfrage war keine Zeit mehr für Dateiuploads, da nicht zwingend notwendig
t. Erstellen der Funktion und Eingabemaske Schadensteammitarbeiter	7h	9h	Einhaltung aller Vorschriften bei Planung nicht ausreichend berücksichtigt
9. Testphase	20h	20h	
d. Testen des Anmeldescreens und Rechteprüfung	4h	6h	Die Tests waren umfangreicher, da es nicht sofort funktioniert hatte und nachträglich Tests zu machen waren
e. Testen der Funktion für Dateiuploads und der Eingabemaske	8h	6h	Tests für den Dateiupload nicht notwendig, da diese Funktion nicht eingebaut wurde
f. Abschlusstest aller Funktionen	8h	8h	
10. Abschluss	14h	13h	
d. Dokumentation für den Anwender	4h	4h	

e. Dokumentation für den Entwickler	8h	7h	Keine Dokumentation für die Datenbanken notwendig, da diese schon existierten
f. Betriebsinterne Dokumentation	2h	2h	
Gesamt	80h	80h	

Tabelle 3: Soll-Ist-Vergleichs

SaS2 – Login Maske der alten Software

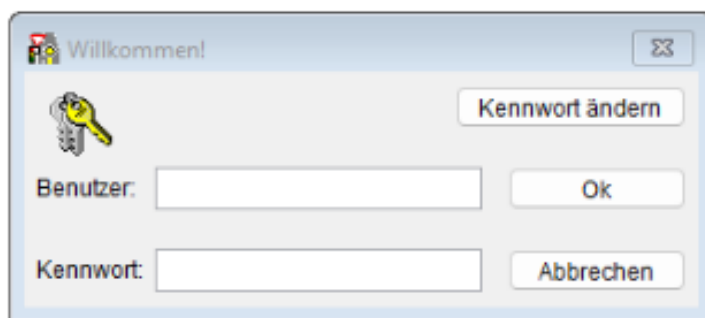


Abbildung 2: SaS2 - Login Maske der alten Software

SaS2 – Übersicht aller eigenen Schadensfälle

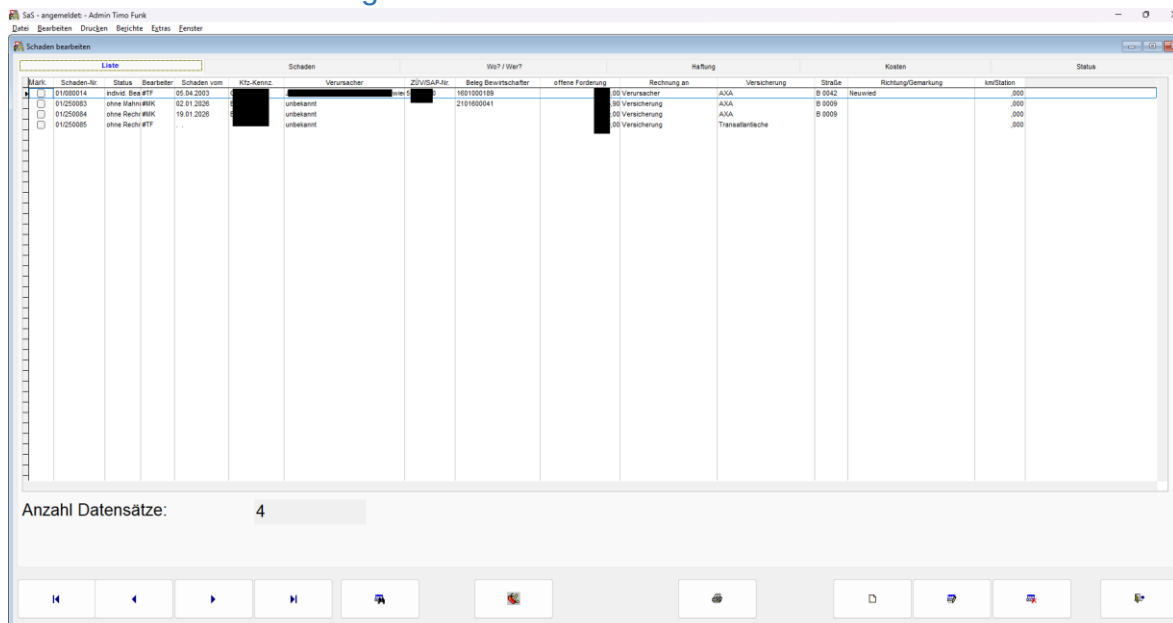
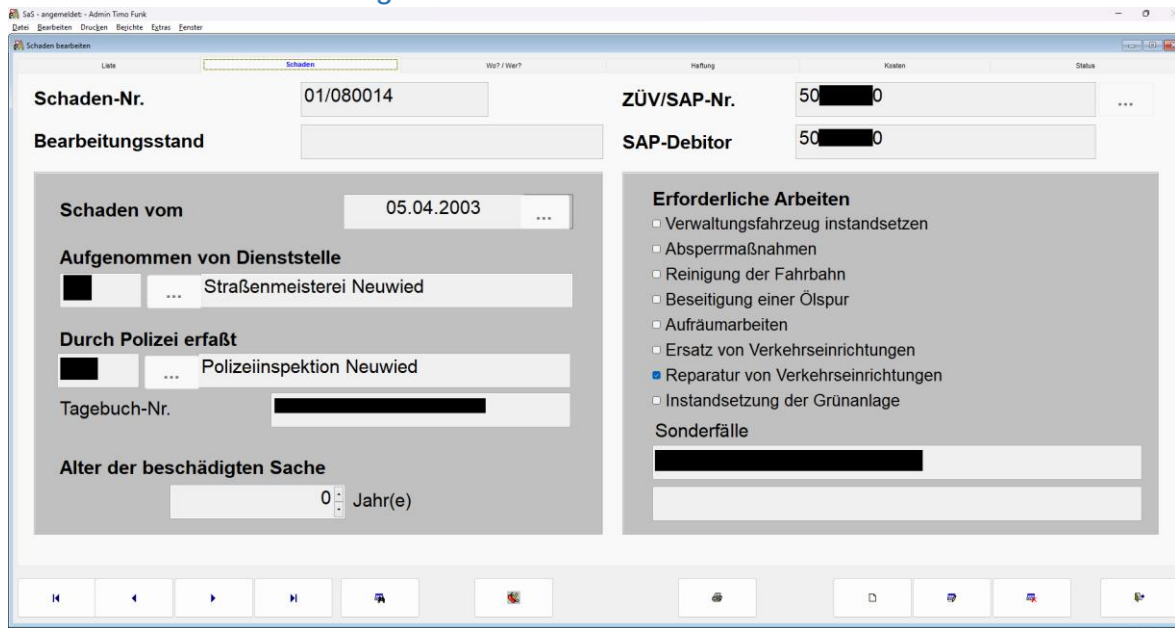


Abbildung 3: SaS2 – Übersicht aller eigenen Schadensfälle

SaS2 – Schadensfall anlegen - Schaden

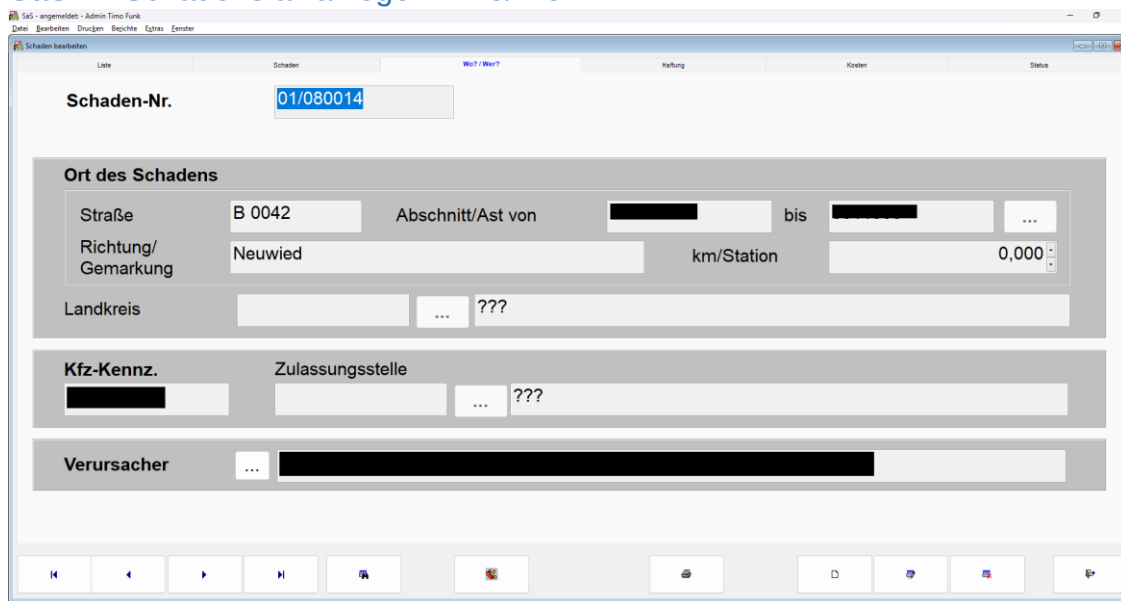


The screenshot shows the 'Schaden' (Damage) form in the SaS2 system. The form is divided into several sections:

- Schaden-Nr.:** 01/080014
- ZÜV/SAP-Nr.:** 50[REDACTED]0
- SAP-Debitor:** 50[REDACTED]0
- Schaden vom:** 05.04.2003
- Aufgenommen von Dienststelle:** [REDACTED] ... Straßenmeisterei Neuwied
- Durch Polizei erfaßt:** [REDACTED] ... Polizeiinspektion Neuwied
- Tagebuch-Nr.:** [REDACTED]
- Alter der beschädigten Sache:** 0 Jahr(e)
- Erforderliche Arbeiten:**
 - ☐ Verwaltungsfahrzeug instandsetzen
 - ☐ Absperurmaßnahmen
 - ☐ Reinigung der Fahrbahn
 - ☐ Beseitigung einer Ölspur
 - ☐ Aufräumarbeiten
 - ☐ Ersatz von Verkehrseinrichtungen
 - ☒ Reparatur von Verkehrseinrichtungen
 - ☐ Instandsetzung der Grünanlage
- Sonderfälle:** [REDACTED]

Abbildung 4: SaS2 - Schadensfall anlegen – Schaden

SaS2 – Schadensfall anlegen - Wo/Wer

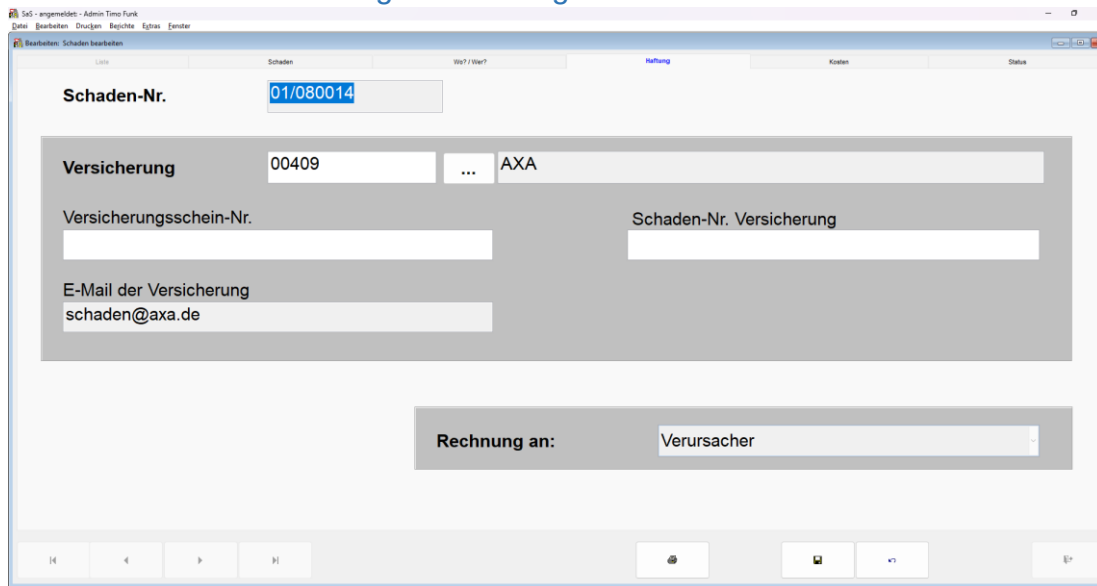


The screenshot shows the 'Wo/Wer' (Where/Who) form in the SaS2 system. The form is divided into several sections:

- Schaden-Nr.:** 01/080014
- Ort des Schadens:**
 - Straße:** B 0042
 - Abschnitt/Ast von:** [REDACTED] bis [REDACTED]
 - Richtung/Gemarkung:** Neuwied
 - km/Station:** 0,000
 - Landkreis:** [REDACTED] ... ???
- Kfz-Kennz.:** [REDACTED]
- Zulassungsstelle:** [REDACTED] ... ???
- Verursacher:** [REDACTED]

Abbildung 5: SaS2 – Schadensfall anlegen - Wo/Wer

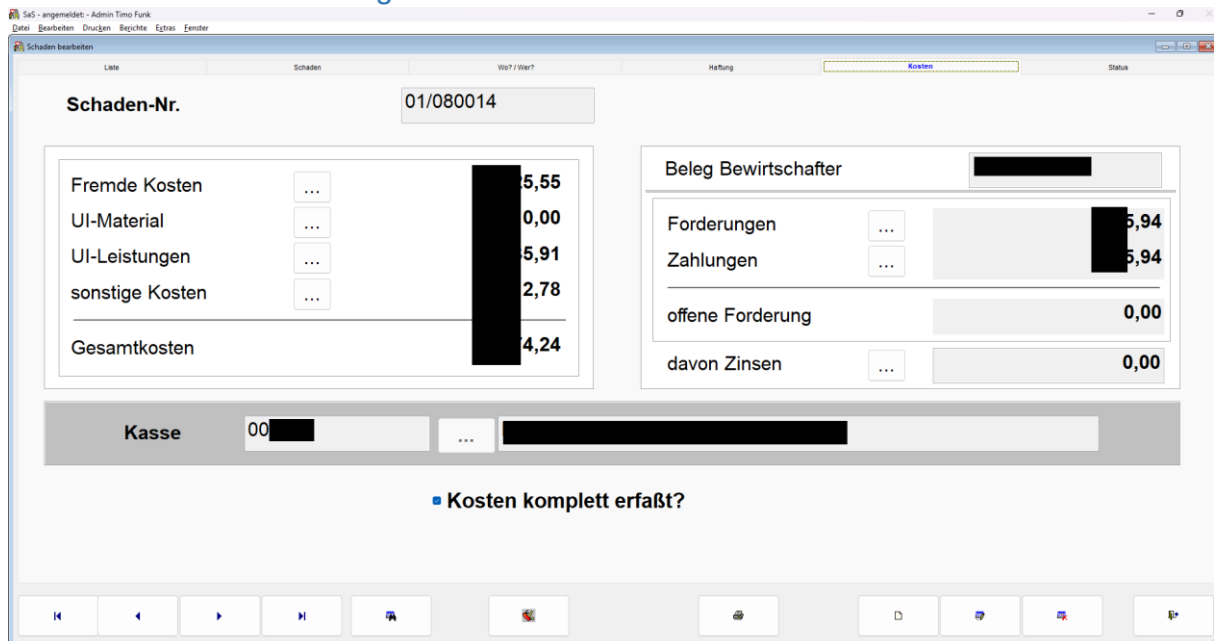
SaS2 – Schadensfall anlegen - Haftung



The screenshot shows the 'Haftung' (Liability) tab of the 'Schaden bearbeiten' (Edit Damage) form. The 'Schaden-Nr.' (Damage No.) is 01/080014. The 'Versicherung' (Insurance) section shows '00409' and 'AXA'. The 'Versicherungsschein-Nr.' (Insurance Policy No.) and 'Schaden-Nr. Versicherung' (Insurance Damage No.) fields are empty. The 'E-Mail der Versicherung' (Insurance Email) is 'schaden@axa.de'. The 'Rechnung an:' (Bill to) field is 'Verursacher' (Causer). The bottom navigation bar includes buttons for 'Liste', 'Schaden', 'Wsp / Wer?', 'Haftung', 'Kosten', and 'Status'.

Abbildung 6: SaS2 – Schadensfall anlegen - Haftung

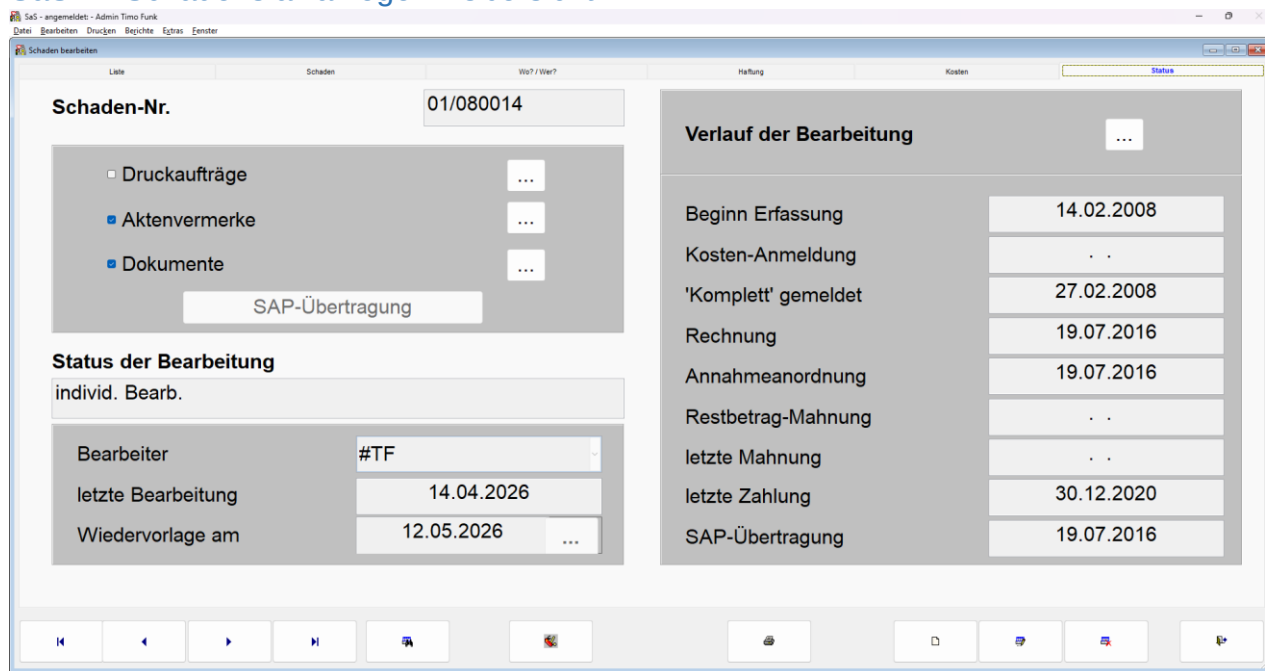
SaS2 – Schadensfall anlegen - Kosten



The screenshot shows the 'Kosten' (Costs) tab of the 'Schaden bearbeiten' (Edit Damage) form. The 'Schaden-Nr.' (Damage No.) is 01/080014. The 'Fremde Kosten' (Third Party Costs) section lists: 'UI-Material' (5,55), 'UI-Leistungen' (0,00), 'sonstige Kosten' (5,91), and 'Gesamtkosten' (2,78). The 'Beleg Bewirtschafter' (Document Manager) section lists: 'Forderungen' (5,94), 'Zahlungen' (5,94), 'offene Forderung' (0,00), and 'davon Zinsen' (0,00). The 'Kasse' (Cash) section shows '00' and a total of 4,24. A checkbox 'Kosten komplett erfasst?' (Costs completely recorded?) is checked. The bottom navigation bar includes buttons for 'Liste', 'Schaden', 'Wsp / Wer?', 'Haftung', 'Kosten', and 'Status'.

Abbildung 7: SaS2 – Schadensfall anlegen - Kosten

SaS2 – Schadensfall anlegen - Übersicht



Schaden-Nr. 01/080014

Verlauf der Bearbeitung

Beginn Erfassung	14.02.2008
Kosten-Anmeldung	. .
'Komplett' gemeldet	27.02.2008
Rechnung	19.07.2016
Annahmeanordnung	19.07.2016
Restbetrag-Mahnung	. .
letzte Mahnung	. .
letzte Zahlung	30.12.2020
SAP-Übertragung	19.07.2016

Status der Bearbeitung
individ. Bearb.

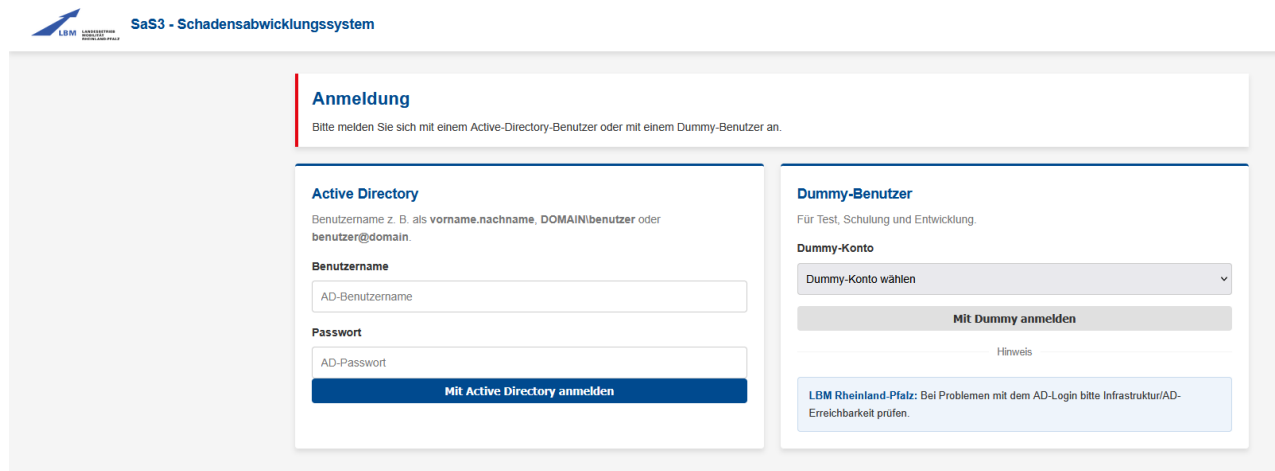
Bearbeiter #TF

letzte Bearbeitung 14.04.2026

Wiedervorlage am 12.05.2026

Abbildung 8: SaS2 – Schadensfall anlegen - Übersicht

SaS3 – Login Maske



Anmeldung
Bitte melden Sie sich mit einem Active-Directory-Benutzer oder mit einem Dummy-Benutzer an.

Active Directory
Benutzername z. B. als vorname.nachname, DOMAIN\benutzer oder benutzer@domain.

Benutzername
AD-Benutzername

Passwort
AD-Passwort

Mit Active Directory anmelden

Dummy-Benutzer
Für Test, Schulung und Entwicklung.

Dummy-Konto
Dummy-Konto wählen

Mit Dummy anmelden

Hinweis
LBM Rheinland-Pfalz: Bei Problemen mit dem AD-Login bitte Infrastruktur/AD-Ereichbarkeit prüfen.

Abbildung 9: SaS3 – Login Maske

SaS3 – Übersicht als Erfasser

SaS3 - Schadensabwicklungssystem Funk, Timo (LBM Zentrale) [Abmelden](#)

Fälle Übersicht

[Neuer Fall](#)

[Home / alle](#)

Fallnummer	Betreff	Dienststelle	Ersteller	Datum	Status	Aktion
01/260004	Schadensfall	Azubi	funkti	13.4.2026	Neu	Ansehen / Bearbeiten

Abbildung 10: SaS3 – Übersicht als Erfasser

SaS3 – Schadensfall anlegen – Allgemeine Daten

Falldetails [Neuer Fall](#) [Speichern](#)

Funk, Timo (LBM Zentrale) - Fälle erfassen (Erfasser) [Blick: eigene Fälle](#) [Bearbeitung möglich](#)

Fall 01/260004 geladen.

Allgemeines

Schaden-Nr.

Betreff / Bezeichnung *

Datum des Schadens *

SAP Debitur

Ausführliche Beschreibung

Status

Neu

Bearbeiterkennung *

SAP Nr.

Erforderliche Arbeiten

☐ Verwaltungsfahrzeug instandsetzen
☐ Reinigung der Fahrbahn
☐ Aufräumarbeiten
☐ Reparatur von Verkehrseinrichtungen

☐ Abspermaßnahmen
☐ Beseitigung einer Ölspur
☐ Ersatz von Verkehrseinrichtungen
☐ Instandsetzung der Grünanlage

Eigener Arbeitspunkt (Freitext)

Aufnahme

Aufnehmende Dienststelle

Polizei Polizei Tagebuch-Nr.

Abbildung 11: SaS3 – Schadensfall anlegen – Allgemeine Daten

SaS3 – Schadensfall anlegen – Unfallort

Unfallort

Straße	Landkreis
Abschnitt von	Abschnitt bis
Richtung / Gemarkung	km / Station

Verursacher

Verursacher (Name) *

Verursacher (Anschrift)

PLZ	Ort
Straße	Hausnummer

PLZ eingeben und passende Orte und Straßen auswählen.

Beteiligte / Fahrzeug

Kfz-Kennzeichen	Zulassungsstelle

Abbildung 12: SaS3 – Schadensfall anlegen – Unfallort

SaS3 – Schadensfall anlegen – Versicherungsauswahl

Versicherung aus Katalog

Versicherung auswählen

-- Versicherung aus Katalog wählen --

Nach der Auswahl werden die Stammdaten der Versicherung übernommen und gesperrt.

Versicherer	Ansprechpartner
Telefon	E-Mail
Straße	
PLZ	Ort
Land	

Abbildung 13: SaS3 – Schadensfall anlegen – Versicherungsauswahl

SaS3 – Schadensfall anlegen – Auswahl Haftung

Haftung

Versicherung	Rechnung an *
AXA	Versicherung ▼
Rechnungsempfänger *	Rechnungs-E-Mail *
AXA	
Rechnungs-Telefon *	
Rechnungsadresse *	
Versicherungsschein-Nr.	Schaden-Nr. (Versicherung)
E-Mail Versicherung	Wiedervorlage am
schaden@axa.com	TT . MM . JJJJ 

Abbildung 14: SaS3 – Schadensfall anlegen – Auswahl Haftung

SaS3 – Schadensfall anlegen – Auswahl Kosten

Kostenberechnung

Material / Leistung auswählen

-- Material/Leistung wählen -- ▼	1 ▲ ▼	Hinzufügen
----------------------------------	-------	-------------------

Position	Einheit	Menge	Satz (€)	Gesamt (€)	Aktion
Mannschaftswagen	Stunde	4.00	9.67 €	38.68 €	Entfernen

Katalog-Gesamt: 38,68 €

Kasse

3000 - Landesbetrieb Mobilität Rheinland-Pfalz ▼	Sonstige Kosten (€)
	0.00 ▲ ▼

Offene Forderung (€)

0.00 ▲ ▼

☐ **Kosten komplett erfasst**

Es fehlen noch: Verursacher, gültige Verursacher-Adresse, Unfallort, Kennzeichen, berechnete Kosten.

Abbildung 15: SaS3 – Schadensfall anlegen – Auswahl Kosten

Sa3 – Administrator – Übersicht mit Notwendigen Funktionen

m

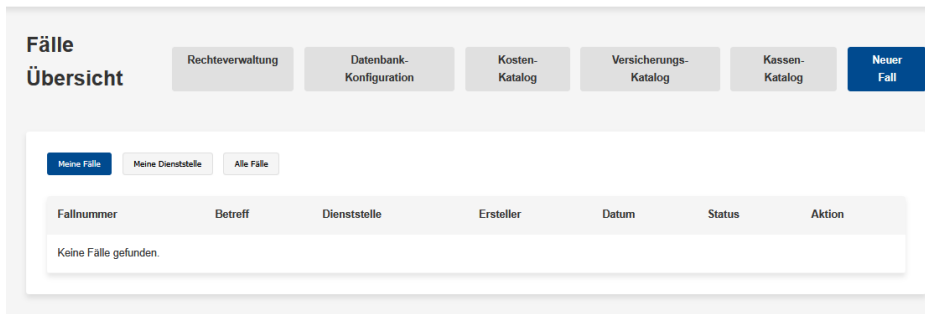


Abbildung 16: Sa3 – Administrator – Übersicht mit Notwendigen Funktionen

Sa3 – Administrator – Rechteverwaltung

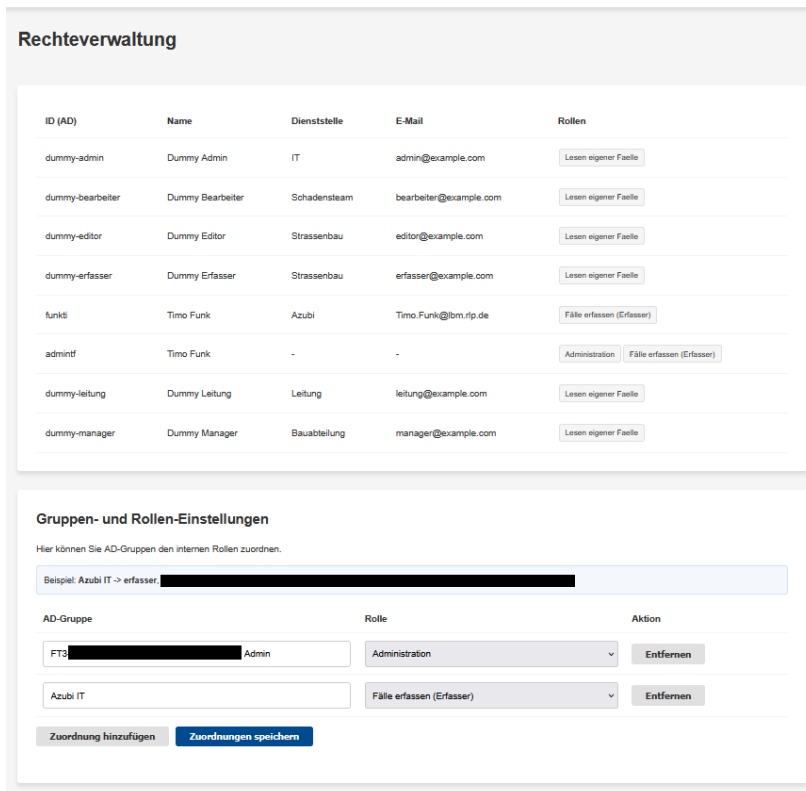


Abbildung 17: Sa3 – Administrator – Rechteverwaltung

SaS3 – Administrator – Kassenkatalog bearbeiten

Kassen-Katalog

Schadensteam und Leitung pflegen hier die verfügbaren Kassen. Beim Erfassen eines Falls kann nur aus diesen Werten ausgewählt werden.

Neue Kasse

Kasse	Status	Aktionen
3000 - Landesbetrieb Mobilität Rheinland-Pfalz	Aktiv	Bearbeiten Deaktivieren
7000 - Bundesrepublik Deutschland	Aktiv	Bearbeiten Deaktivieren

Abbildung 18: SaS3 – Administrator – Kassenkatalog bearbeiten

SaS3 – Administrator – Kostenkatalog bearbeiten

Kosten-Katalog

Verwalten Sie Materialien und Leistungen für die Kostenberechnung in Schadensfällen.

Katalog: Verwalten Sie hier die verfügbaren Materialien und Leistungen mit ihren Kosten. Diese können beim Erfassen eines Schadensfalls zur Kostenberechnung ausgewählt werden.

Neuer Katalog-Eintrag

Bezeichnung *

Kategorie

Beschreibung

Einheit

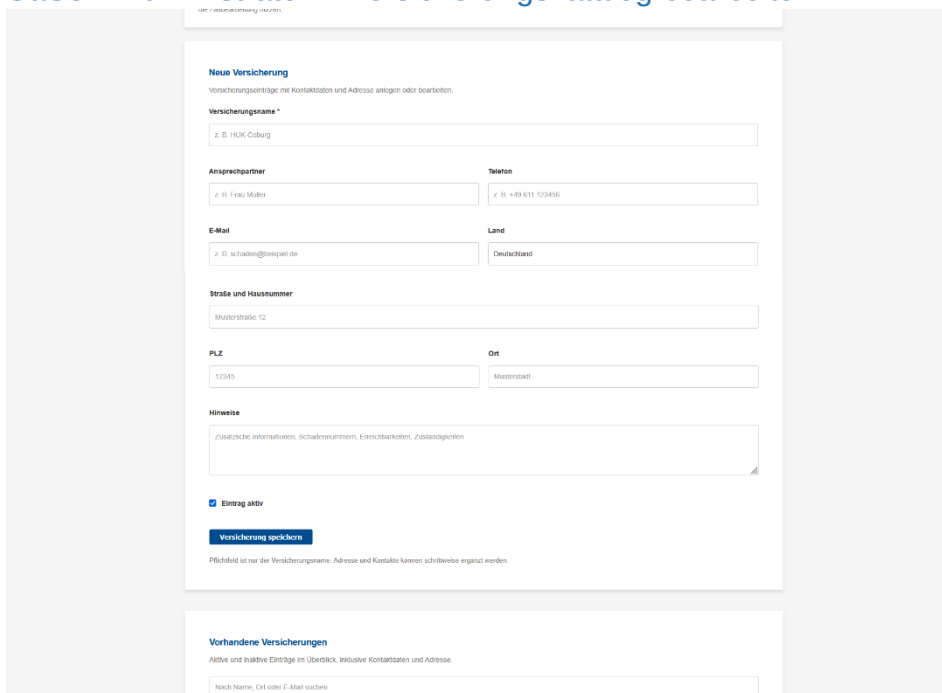
Satz (€) *

Katalog-Einträge

Bezeichnung	Kategorie	Einheit	Satz (€)	Status	Aktionen
Mannschaftswagen	Fahrzeug	Stunde	9.67 €	✓ Aktiv	Bearbeiten Löschen

Abbildung 19: SaS3 – Administrator – Kostenkatalog bearbeiten

SaS3 – Administrator – Versicherungskatalog bearbeiten



Neue Versicherung
Versicherungseinträge mit Kontaktdaten und Adresse anlegen oder bearbeiten.

Versicherungsname *
z. B. HUK Coburg

Ansprechpartner
z. B. Frau Müller

Telefon
z. B. +49 611 123456

E-Mail
z. B. schaden@beispiel.de

Land
Deutschland

Straße und Hausnummer
Musterstraße 12

PLZ
12345

Ort
Musterstadt

Hinweise
Zusätzliche Informationen, Schadensnummern, Erreichbarkeiten, Zuordnungen

☒ Eintrag aktiv

Versicherung speichern

Pflichtfeld ist nur der Versicherungsname. Adresse und Kontakte können schrittweise ergänzt werden.

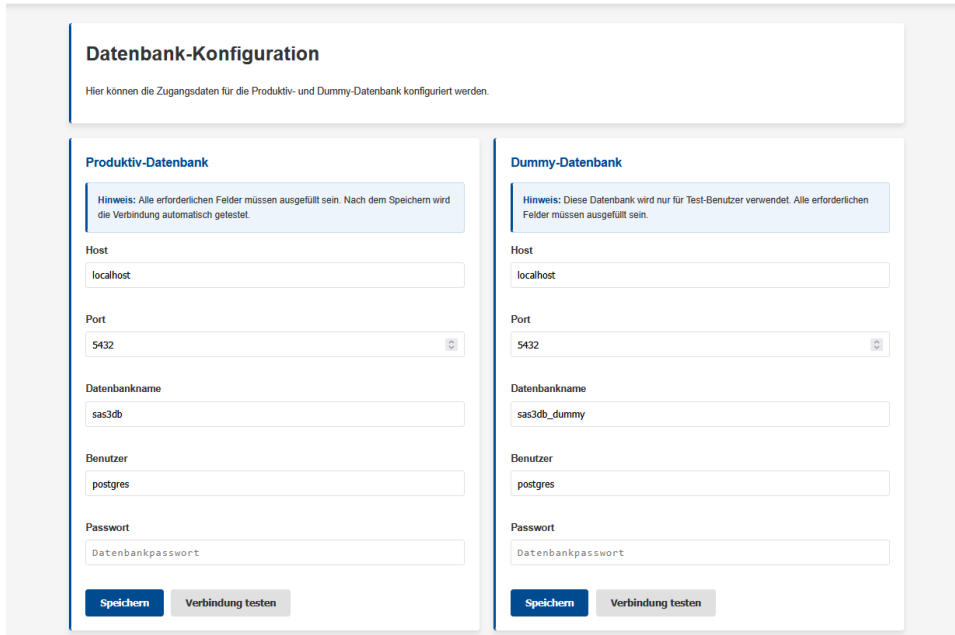
Vorhandene Versicherungen
Active und inaktive Einträge im Überblick, inklusive Kontaktdaten und Adresse.

Nach Name, Ort oder E-Mail suchen

Abbildung 20: SaS3 – Administrator – Versicherungskatalog bearbeiten

SaS3 – Administrator – Datenbankverbindung konfigurieren

ation



Datenbank-Konfiguration
Hier können die Zugangsdaten für die Produktiv- und Dummy-Datenbank konfiguriert werden.

Produktiv-Datenbank

Hinweis: Alle erforderlichen Felder müssen ausgefüllt sein. Nach dem Speichern wird die Verbindung automatisch getestet.

Host
localhost

Port
5432

Datenbankname
sas3db

Benutzer
postgres

Passwort
Datenbankpasswort

Speichern **Verbindung testen**

Dummy-Datenbank

Hinweis: Diese Datenbank wird nur für Test-Benutzer verwendet. Alle erforderlichen Felder müssen ausgefüllt sein.

Host
localhost

Port
5432

Datenbankname
sas3db_dummy

Benutzer
postgres

Passwort
Datenbankpasswort

Speichern **Verbindung testen**

Abbildung 21: SaS3 – Administrator – Datenbankverbindung konfigurieren

SaS3 – Übersicht mit Datenbankverbindungsfehler

Meine Fälle						
Fallnummer	Betreff	Dienststelle	Ersteller	Datum	Status	Aktion
Fehler beim Laden der Fälle						

Abbildung 22: SaS3 – Übersicht mit Datenbankverbindungsfehler

SaS3 – UML-Diagramm

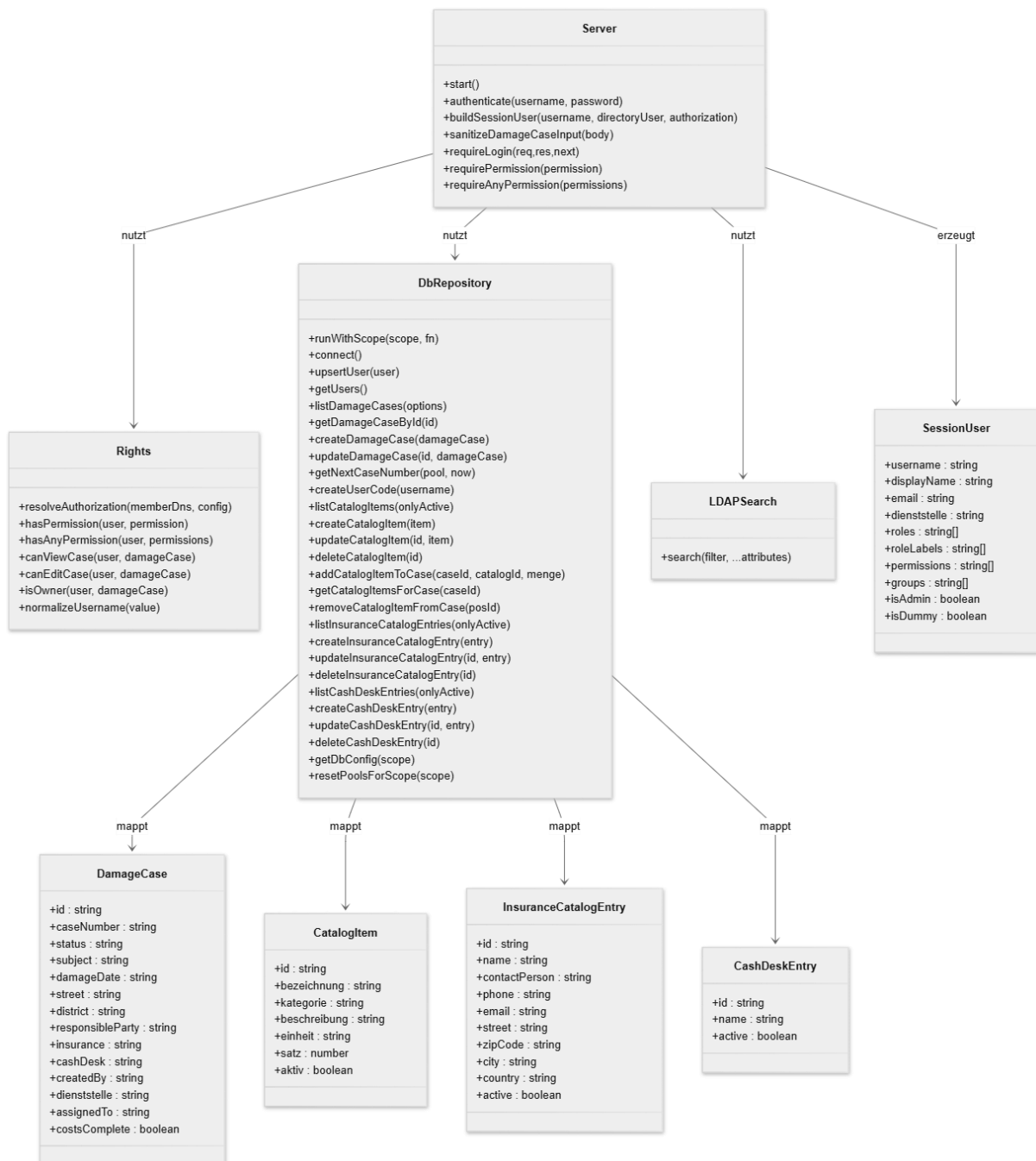


Abbildung 23: SaS3 – UML-Diagramm

Anwenderdokumentation

SaS3

1. Zweck der Anwendung

SaS3 unterstützt die Erfassung, Bearbeitung und den Abschluss von Schadensfällen an Straßeneinrichtungen.

Die Anwendung bietet:

- Anmeldung mit Active Directory oder Dummy-Benutzer (je nach Umgebung)
- Übersicht über eigene, dienststellenbezogene oder alle sichtbaren Fälle
- Erfassung und Bearbeitung von Schadensfällen
- Kataloggestützte Kostenverwaltung
- Verwaltung von Versicherungs- und Kassenstammdaten
- Rechteverwaltung und Datenbank-Konfiguration (nur berechtigte Benutzer)

2. Rollen und Berechtigungen

Welche Funktionen sichtbar sind, hängt von Ihrer Rolle ab.

Typische Unterschiede:

- Erfasser: eigene Fälle anlegen und bearbeiten
- Bearbeiter/Team: Teamfälle bearbeiten, ggf. an Leitung weiterleiten
- Leitung: freigabepflichtige Fälle prüfen und abschließen
- Admin: zusätzliche Administrationsseiten (Rechteverwaltung, DB-Konfiguration)

Hinweis: Nicht jeder Benutzer sieht alle Schaltflächen. Fehlende Buttons sind in der Regel berechtigungsbedingt.

3. Anmeldung

3.1 Aufruf

1. Öffnen Sie die Anmeldeseite der Anwendung.
2. Wählen Sie eine der zwei Anmeldearten:
 - Active Directory: Benutzername + Passwort
 - Dummy-Benutzer: Auswahl aus der Liste (Test/Schulung)

3.2 Benutzername für AD

Folgende Formate sind möglich:

- vorname.nachname
- DOMAIN\benutzer
- benutzer@domain

3.3 Nach erfolgreicher Anmeldung

Sie werden automatisch zur Übersicht weitergeleitet.

4. Startseite: Fälle-Übersicht

Die Übersicht ist Ihre zentrale Arbeitsseite.

Elemente:

- Filter für Sichtbereich: Meine Fälle, Dienststelle, Alle Fälle (rollenabhängig)
- Tabelle mit Fallnummer, Betreff, Dienststelle, Ersteller, Datum, Status
- Schaltfläche Neuer Fall (nur mit entsprechender Berechtigung)
- Links zu Katalogen, Rechtsverwaltung, Datenbank-Konfiguration (rollenabhängig)

Typischer Ablauf:

1. Sichtbereich auswählen.
2. Fall in der Tabelle anklicken, um Details zu öffnen.
3. Oder Neuen Fall anlegen.

5. Schadensfall anlegen und bearbeiten

5.1 Neuen Fall starten

1. In der Übersicht auf Neuer Fall klicken.
2. Die Anwendung reserviert eine Fallnummer.
3. Formular ausfüllen und speichern.

5.2 Wichtige Eingabebereiche

- Allgemeines: Betreff, Datum, Beschreibung
- Unfall-/Ortsdaten: Straße, Abschnitt, Richtung, km-Angabe
- Verursacherdaten: Name, Kennzeichen, Adresse
- Versicherungs- und Rechnungsdaten
- Kostenangaben: Katalogpositionen, sonstige Kosten, offene Forderung

5.3 Automatische Unterstützungen

- Kennzeichen-Lookup kann Versicherungs- und Zulassungsdaten ergänzen.
- Straßen-Lookup kann die Dienststelle/Region unterstützen.
- Adressprüfung unterstützt bei der Verursacheradresse.

Wichtig: Wenn die Adresse nicht als gültig erkannt wird, kann der Fall nicht mit ungültiger Adresse gespeichert werden.

5.4 Rechnungsempfänger

Wählbare Empfängerarten:

- Verursacher
- Versicherung
- Andere Stelle

Bei Andere Stelle müssen die Kontaktdaten vollständig ausgefüllt sein.

5.5 Kosten komplett erfasst

Die Option Kosten komplett erfasst kann erst gesetzt werden, wenn alle Voraussetzungen erfüllt sind, insbesondere:

- Verursacher vorhanden
- gültige Verursacheradresse
- Versicherung gesetzt
- Kasse gesetzt
- Unfallort gesetzt
- Kennzeichen gesetzt
- mindestens eine Kostenbasis vorhanden (Katalogsumme, sonstige Kosten oder offene Forderung)

5.6 Speichern

1. Auf Speichern klicken.
2. Bei Erfolg erscheint eine Bestätigung.
3. Der Fall wird in der Liste aktualisiert.

6. Workflow: Weiterleiten und Freigeben

Je nach Rolle und Status stehen zusätzliche Aktionen zur Verfügung.

- An Leitung weiterleiten:
 - sichtbar für berechnete Mitarbeiter

- typischerweise bei Status Team
- Prüfen & Freigeben:
 - sichtbar für berechnete Leitung
 - typischerweise bei Status Leitung

Nach Aktion wird der Fallstatus entsprechend fortgeschrieben.

7. Kataloge bedienen

7.1 Kosten-Katalog

Zweck: Pflege von Material-/Leistungspositionen mit Satz, Einheit, Kategorie und Aktiv-Status.

Typische Aktionen:

- neue Position anlegen
- bestehende Position bearbeiten
- Position deaktivieren/löschen

7.2 Versicherungs-Katalog

Zweck: Pflege von Versicherungsstammdaten für die Fallbearbeitung.

Typische Aktionen:

- Eintrag anlegen
- Eintrag bearbeiten
- Aktiv/Inaktiv schalten
- über Suche filtern

7.3 Kassen-Katalog

Zweck: Pflege der verfügbaren Kassen, aus denen im Fall ausgewählt wird.

Typische Aktionen:

- Kasse hinzufügen
- Kasse bearbeiten
- Kasse aktivieren/deaktivieren

8. Rechteverwaltung (Admin)

In der Rechteverwaltung können AD-Gruppen internen Rollen zugeordnet werden.

Ablauf:

1. Gruppen-/Rollen-Zuordnung hinzufügen oder bearbeiten.
2. Ungültige/alte Zuordnung entfernen.
3. Zuordnungen speichern.

Zusätzlich wird eine Benutzerliste mit Rollen angezeigt.

9. Datenbank-Konfiguration (Admin)

Hier werden Verbindungen für zwei Umgebungen gepflegt:

- Produktiv-Datenbank
- Dummy-Datenbank

Ablauf je Bereich:

1. Host, Port, Datenbankname, Benutzer, Passwort eintragen.
2. Verbindung testen.
3. Konfiguration speichern.

Hinweis: Fehlermeldungen beim Test weisen meist auf Netzwerk, Zugangsdaten oder Erreichbarkeit des DB-Servers hin.

10. Abmelden

Über Abmelden in der Kopfzeile wird die Sitzung beendet und zur Anmeldeseite zurückgeleitet.

11. Häufige Probleme und Lösungen

Problem: Anmeldung fehlgeschlagen

- Benutzernameformat prüfen
- Passwort prüfen
- AD-Erreichbarkeit prüfen

Problem: Speichern nicht möglich

- Pflichtfelder prüfen
- Rechnungsempfänger vollständig ausfüllen
- Verursacheradresse mit gültigem Vorschlag übernehmen

Problem: Schaltfläche fehlt

- Berechtigung/Rolle prüfen
- Fallstatus prüfen (z. B. Weiterleiten/Freigeben)

Problem: Keine Fälle sichtbar

- Filterbereich prüfen (Eigene/Dienststelle/Alle)
- prüfen, ob Fälle im gewählten Bereich vorhanden sind

Entwicklerdokumentation für SaS3

Stand der Dokumentation

- Letzte inhaltliche Aktualisierung: 2026-04-14
- Quelle: aktueller Code-Stand im Repository
- Ergänzendes Fachbegriff-Glossar aus der Projektdokumentation: `doku/Glossar.md`

1. Ziel und Umfang

Diese Dokumentation beschreibt die technische Architektur und die wichtigsten Funktionen der Anwendung SaS3 auf Basis des aktuellen Codes.

Zielgruppe:

- Backend-Entwickler
- Frontend-Entwickler
- Betrieb/Administration

Abgedeckte Kernbereiche:

- Login, Session und Autorisierung
- Schadensfall-Workflow
- Datenbanklayer und Schema
- Kataloge (Material, Versicherung, Kasse)
- Admin-Funktionen (Rechte, DB-Konfiguration)
- Frontend-Seiten und deren API-Anbindung

2. Systemüberblick

Zentrale Komponenten:

- `server.mjs`: Express-Server, Routing, Login, Workflow, Guards
- `src/db.js`: PostgreSQL-Zugriffe, Schema-Setup, Mapping-Funktionen
- `src/rights.js`: Rollenmodell, Rechteableitung, Sicht-/Editierprüfungen
- `src/LDAPSearch.js`: LDAP-Suchen für AD-Authentifizierung und Gruppen
- `src/create-damage-case.js`: Client-Logik für Schadensfallmaske
- `public/*.html`: UI-Seiten
- `config/config.json`: Server-, LDAP-, DB-, Rollen- und Testuser-Konfiguration

Laufzeitmodus:

- Scope `main`: Produktiv-DB
- Scope `dummy`: Dummy/Test-DB

Der Scope wird pro Request gesetzt:

```
app.use((req, _res, next) => {  
  const scope = req.session?.user?.isDummy ? "dummy" : "main";  
  db.runWithScope(scope, next);  
});
```

3. Start und Betrieb

3.1 Startkommando

```
{  
  "scripts": {  
    "start": "node server.mjs"  
  }  
}
```

3.2 HTTP/HTTPS-Start mit Zertifikatserkennung

```
function start() {  
  const certPath = "/etc/lbm/ssl/privkey.pem";  
  const fullchainPath = "/etc/lbm/ssl/fullchain.pem";  
  
  const hasCerts = fs.existsSync(certPath);  
  
  if (hasCerts) {  
    https.createServer(sslOptions, app).listen(config.server.port, () => {  
      connectDatabaseWithRetry();  
    });  
  } else {  
    http.createServer(app).listen(config.server.port, () => {  
      connectDatabaseWithRetry();  
    });  
  }  
}
```

3.3 DB-Connect mit Retry

```
async function connectDatabaseWithRetry() {  
  const retryDelayMs = 15000;  
  
  try {  
    await db.connect();  
    console.log("Datenbankverbindung hergestellt.");  
  } catch (error) {  
    setTimeout(connectDatabaseWithRetry, retryDelayMs);  
  }  
}
```

4. Authentifizierung und Session

4.1 Login-Typen

- Dummy-Login über `config.testUsers`
- AD-Login über LDAP mit Benutzername + Passwort

4.2 Benutzername-Normalisierung

```
function normalizeLoginUsername(value) {  
  const raw = String(value || "").trim();  
  const withoutDomainPrefix = raw.includes("\\") ? raw.split("\\").pop() : raw;  
  const withoutUpnSuffix = withoutDomainPrefix.includes("@")  
    ? withoutDomainPrefix.split("@")[0]  
    : withoutDomainPrefix;  
  
  return withoutUpnSuffix.trim();  
}
```

Akzeptierte Eingaben:

- domain\\user
- user@domain
- user

4.3 AD-Credential-Prüfung

```
const credentialCheck = new LDAPSearch(  
  config.ldap.realm,  
  normalizedUsername,  
  normalizedPassword,  
);  
  
const filter = config.ldap.userFilter.replace("{username}", normalizedUsername);  
const authResults = await credentialCheck.search(filter, "dn");
```

4.4 Session-Regeneration (Fixation-Schutz)

```
await new Promise((resolve, reject) => {  
  req.session.regenerate((error) => {  
    if (error) return reject(error);  
    req.session.user = result.user;  
    resolve();  
  });  
});
```

4.5 Login-Endpunkte

- GET /api/auth-options: Dummy-Accounts für Loginseite
- POST /login: Login
- GET /api/me: Session-User
- GET /logout: Session beenden

5. Rollen- und Rechtesystem

Rechte werden aus LDAP-Gruppen und `config.rights` berechnet.

5.1 Rollenvererbung

```
function expandRole(roleName, roleDefinitions, visited = new Set()) {
  if (!roleName || visited.has(roleName)) return [];

  visited.add(roleName);
  const definition = roleDefinitions[roleName] || {};
  const inheritedRoles = asArray(definition.inherits);
  const permissions = new Set(asArray(definition.permissions));

  inheritedRoles.forEach((inheritedRole) => {
    expandRole(inheritedRole, roleDefinitions, visited).forEach((permission) => {
      permissions.add(permission);
    });
  });

  return Array.from(permissions);
}
```

5.2 Kernfunktionen in `src/rights.js`

- `resolveAuthorization(memberDns, config)`
- `hasPermission(user, permission)`
- `hasAnyPermission(user, permissions)`
- `canViewCase(user, damageCase)`
- `canEditCase(user, damageCase)`
- `isOwner(user, damageCase)`

5.3 Statusabhängige Sichtbarkeit/Bearbeitung

```
if (hasPermission(user, "approve_case") && status === "Leitung") {
  return true;
}
```

Das Rechtekonzept ist nicht nur rollenbasiert, sondern auch workflow-/statusbasiert.

6. Datenbankschicht (`src/db.js`)

6.1 Kernprinzipien

- Pool-Caching pro Scope (main, dummy)
- automatisches Schema-Setup beim ersten Zugriff
- Mapping DB -> API-Objekt
- Soft-Delete bei Katalogen/Positionen

6.2 Scope via `AsyncLocalStorage`

```
const dbScopeStorage = new AsyncLocalStorage();

function runWithScope(scope, fn) {
  const normalizedScope = scope === "dummy" ? "dummy" : "main";
  return dbScopeStorage.run({ scope: normalizedScope }, fn);
}
```

6.3 Wichtige Tabellen

- SYSBENUTZER
- DATSCHADENSFAELLE
- SYSKATALOG
- SFOKATALOGPOSITIONEN
- SYSVERSICHERUNGEN
- SYSKASSEN

6.4 Schlüsselfunktionen

Benutzer:

- upsertUser(user)
- getUsers()

Schadensfälle:

- createDamageCase(damageCase)
- listDamageCases(options)
- getDamageCaseById(id)
- updateDamageCase(id, damageCase)
- getNextCaseNumber(pool, now)
- createUserCode(username)

Material-/Leistungskatalog:

- createCatalogItem(item)
- listCatalogItems(onlyActive)
- updateCatalogItem(id, item)
- deleteCatalogItem(id)

Katalogpositionen am Fall:

- addCatalogItemToCase(caseId, catalogId, menge)
- getCatalogItemsForCase(caseId)
- removeCatalogItemFromCase(posId)

Versicherungskatalog:

- createInsuranceCatalogEntry(entry)
- listInsuranceCatalogEntries(onlyActive)
- updateInsuranceCatalogEntry(id, entry)
- deleteInsuranceCatalogEntry(id)

Kassenkatalog:

- createCashDeskEntry(entry)

- listCashDeskEntries (onlyActive)
- updateCashDeskEntry (id, entry)
- deleteCashDeskEntry (id)
- hasActiveCashDesk (name)

Konfiguration:

- getDbConfig (scope)
- resetPoolsForScope (scope)

6.5 Beispiel Fallnummernlogik

```
async function getNextCaseNumber(pool, now) {
  const yearToken = String(now.getFullYear()).slice(-2);
  const likePattern = `01/${yearToken}%`;

  const result = await pool.query(
    'SELECT MAX("SFANUMMER") AS "maxNumber" FROM "DATSCHADENSFAELLE" WHERE "SFANUMMER" LIKE $1',
    [likePattern],
  );

  const currentSequence = result.rows[0].maxNumber
    ? parseInt(String(result.rows[0].maxNumber).slice(-4), 10)
    : 0;

  return `01/${yearToken}${String(currentSequence + 1).padStart(4, "0")}`;
}
```

7. Schadensfall-Workflow

Status im Backend:

- Neu
- Team
- Leitung
- Abgeschlossen

7.1 Anlegen

Beim Anlegen wird auf Basis `costsComplete` entschieden:

```
if (payload.costsComplete) {
  payload.status = "Team";
} else {
  payload.status = "Neu";
}
```

7.2 Voraussetzungen für "Kosten komplett"

Servervalidierung:

```
function isValidCostsCompletePrerequisites(damageCase) {
  const hasResponsibleParty = Boolean(String(damageCase.responsibleParty || "").trim());
  const isValidResponsiblePartyAddress = Boolean(String(damageCase.responsiblePartyAddress || "").trim());
  const hasInsurance = Boolean(String(damageCase.insurance || "").trim());
  const hasCashDesk = Boolean(String(damageCase.cashDesk || "").trim());
  const hasAccidentLocation = Boolean(String(damageCase.street || "").trim());
  const hasPlateNumber = Boolean(String(damageCase.plateNumber || "").trim());

  return (
    hasResponsibleParty &&
    isValidResponsiblePartyAddress &&
    hasInsurance &&
    hasCashDesk &&
    hasAccidentLocation &&
    hasPlateNumber
  );
}
```

7.3 Weiterleitung/Freigabe

- forwardToLeitung + **Recht** forward_to_leitung -> **Status** Leitung
- approve + **Recht** approve_case -> **Status** Abgeschlossen

7.4 Gesperrte Kostenfelder nach Abschluss

Nach costsComplete = true sind relevante Kostenfelder nicht mehr änderbar.

8. Lookup- und Hilfsfunktionen

8.1 Fahrzeug-Lookup

- GET /api/lookup/vehicle?plate=...
- nutzt interne VEHICLE_DATA

8.2 Strassen- und Adress-Lookup

- GET /api/lookup/street
- GET /api/lookup/address

Internet-Lookup über Nominatim mit Timeout/Fallback:

```
const controller = new AbortController();
const timeoutId = setTimeout(() => controller.abort(), 5000);
```

Bei transienten Fehlern wird auf lokale Fallback-Daten gearbeitet (wo verfügbar).

9. API-Referenz nach Bereich

9.1 UI-Seiten

- GET /login

- GET /overview
- GET /create-damage-case
- GET /rights
- GET /catalog
- GET /insurance-catalog
- GET /cash-desk-catalog
- GET /db-config

9.2 Schadensfälle

- GET /api/damage-cases?scope=own|department|all
- GET /api/damage-cases/:id
- POST /api/damage-cases
- PUT /api/damage-cases/:id

9.3 Katalogpositionen am Fall

- GET /api/damage-cases/:id/catalog-items
- POST /api/damage-cases/:id/catalog-items
- DELETE /api/damage-cases/:id/catalog-items/:posId

9.4 Material-/Leistungskatalog

- GET /api/catalog (aktiv)
- GET /api/admin/catalog
- POST /api/admin/catalog
- PUT /api/admin/catalog/:id
- DELETE /api/admin/catalog/:id

9.5 Versicherungskatalog

- GET /api/insurance-catalog (aktiv)
- GET /api/admin/insurance-catalog
- POST /api/admin/insurance-catalog
- PUT /api/admin/insurance-catalog/:id
- DELETE /api/admin/insurance-catalog/:id

9.6 Kassenkatalog

- GET /api/cash-desks (aktiv)
- GET /api/admin/cash-desks
- POST /api/admin/cash-desks
- PUT /api/admin/cash-desks/:id
- DELETE /api/admin/cash-desks/:id

9.7 Rechte-/Benutzerverwaltung

- GET /api/users
- GET /api/rights/group-mappings
- PUT /api/rights/group-mappings

9.8 DB-Konfiguration

- GET /api/db-config
- PUT /api/db-config
- POST /api/db-config/test

10. Frontend-Bestandteile

10.1 Seiten

- public/login.html: **AD + Dummy Login**
- public/overview.html: **Fallliste, Scopes, Admin-Links**
- public/create_damage_case.html: **Detailmaske inkl. Katalogpositionen**
- public/rights.html: **Rechteverwaltung**
- public/catalog.html: **Material/Leistung**
- public/insurance-catalog.html: **Versicherungen**
- public/cash-desk-catalog.html: **Kassen**
- public/db-config.html: **DB-Einstellungen**

10.2 Zentrale Client-Logik (src/create-damage-case.js)

State-Objekt enthält u. a.:

- cases
- catalogItems
- insuranceCatalog
- cashDeskCatalog
- responsiblePartyAddressValid

Beispiel Kostenvoraussetzungen im Frontend:

```
function isValidCostsCompletePrerequisites() {  
  const hasResponsibleParty = Boolean(elements.responsibleParty.value.trim());  
  const hasInsurance = Boolean(elements.insurance.value.trim());  
  const hasCashDesk = Boolean(elements.cashDesk.value.trim());  
  const hasAccidentLocation = Boolean(elements.street.value.trim());  
  const hasPlateNumber = Boolean(elements.plateNumber.value.trim());  
  
  return (  
    hasResponsibleParty &&  
    hasInsurance &&  
    hasCashDesk &&  
    hasAccidentLocation &&  
    hasPlateNumber  
  );  
}
```

11. Guarding und Sicherheit

Wichtige Guards im Server:

```
function requireLogin(req, res, next) {  
  if (req.session && req.session.user) return next();  
  if (req.path.startsWith("/api/")) return res.status(401).json({ error: "Nicht angemeldet" });  
  res.redirect("/login");  
}  
  
function requirePermission(permission) {  
  return (req, res, next) => {  
    if (rights.hasPermission(req.session?.user, permission)) return next();  
    res.status(403).json({ error: "Zugriff verweigert", missingPermission: permission });  
  };  
}
```

Technische Hinweise:

- Session-Cookie aktuell mit `secure: false` (lokal ohne HTTPS möglich)
- produktiv sollte HTTPS mit gültigen Zertifikaten aktiv sein
- Fehler werden serverseitig geloggt, API liefert gezielte Fehlermeldungen

12. Wichtige Entwicklungsregeln

1. Rechte nur in `src/rights.js` erweitern, nicht in Endpunkten duplizieren.
2. SQL-Zugriffe nur in `src/db.js` pflegen.
3. Neue Workflow-Status immer in beiden Schichten anpassen:
 - Rechteprüfung (`canViewCase`, `canEditCase`)
 - API-Workflow (`POST/PUT /api/damage-cases`)
4. Neue Felder für Schadensfälle in drei Stellen synchron halten:
 - `sanitizeDamageCaseInput` in `server.mjs`
 - Persistenz in `src/db.js`
 - Form-Bindings in `src/create-damage-case.js`
5. Kataloge bevorzugt soft-löschen, damit Historie konsistent bleibt.

13. Typischer End-to-End-Ablauf

1. Login über `/login` (Dummy oder AD).
2. Rechteableitung über LDAP-Gruppen und Rollenmapping.
3. Session-User wird gesetzt, Benutzer via `db.upsertUser` synchronisiert.
4. Übersicht lädt Fälle via `/api/damage-cases`.
5. Detailseite lädt Fall und Katalogpositionen.
6. Bei Änderungen greifen Status-/Rechte-/Validierungsregeln serverseitig.
7. Katalog-, Versicherungs- und Kassenstammdaten werden über Admin-APIs gepflegt.

14. Kurzfazit

SaS3 ist als klare 3-Schichten-Struktur umgesetzt:

- API/Workflow in `server.mjs`
- Rechte in `src/rights.js`
- Datenhaltung in `src/db.js`

Die wichtigsten Erweiterungspunkte sind bereits zentralisiert. Dadurch lassen sich neue Fachregeln, Felder und Rollen ohne verteilte Sonderlogik implementieren.

15. Offene Punkte (bewusst nicht umgesetzt)

Die folgenden Themen sind im aktuellen Stand als nächste Ausbaustufe vorgesehen:

- Dateiupload inkl. revisionssicherer Ablage
- SAP-Schnittstelle für Rechnungsübergabe und Rückmeldung zum Zahlungseingang

Hinweis für die technische Umsetzung:

- Beide Themen betreffen API, Berechtigungslogik, Persistenz und Betriebskonzept.
- Bei Umsetzung sollten Integrations- und Regressionstests für den kompletten Workflow ergänzt werden.

Prüfprotokoll White-Box-Tests (C1) - SaS3

1. Rahmenbedingungen

- **Projekt:** SaS3 (Neuentwicklung der Schadenserfassung)
- **Testart:** Ausschließlich White-Box-Tests
- **Überdeckung:** C1 (Entscheidungs-/Zweigüberdeckung)
- **Kriterium:** Jede relevante `if`-Abfrage wurde mindestens einmal ausgeführt
- **Testdatum:** 16.04.2026
- **Testumgebung:** Lokale Entwicklungsumgebung mit Dummy- und Main-Scope

2. Testgegenstand

Getestet wurden die Verzweigungen in folgenden Komponenten:

- `server.mjs` (Authentifizierung, Guards, Workflow, API-Validierung)
- `src/rights.js` (Rollen- und Rechteentscheidungen)
- `src/create-damage-case.js` (Frontend-Validierung und Ablaufsteuerung)

3. Prüfprotokoll Backend (server.mjs) & Infrastruktur

ID	Bereich	Verzweigung (<code>if</code>)/ Testziel	Testdurchführung	Erwartetes Ergebnis	Ist-Ergebnis	Status
1	Login	<code>if (!normalizedUsername)</code>	Login ohne Benutzername	HTTP 400 Fehlerhinweis	Fehlerhinweis ausgegeben	Bestanden
2	Login	<code>if (testUser)</code>	Dummy-Benutzer genutzt	Dummy-Login erfolgreich	Session gesetzt	Bestanden
3	Login	<code>if (!normalizedPassword)</code>	AD-Login ohne Passwort	HTTP 400 Fehlerhinweis	Fehlerhinweis ausgegeben	Bestanden
4	Login	AD-Anbindung (1. Versuch)	Login mit AD-Benutzer	Zugriff auf Anwendung	Fehlerhafte Gruppenkonfiguration im AD	Nicht bestanden

ID	Bereich	Verzweigung (<code>if</code>) / Testziel	Testdurchführung	Erwartetes Ergebnis	Ist-Ergebnis	Status
5	Login	AD-Anbindung (2. Versuch)	Login nach Korrektur	Zugriff auf Anwendung	Login erfolgreich	Bestanden
6	Login	<code>if (!authorization.isMember)</code>	Benutzer ohne passende Gruppen	HTTP 403	Zugriff verweigert	Bestanden
7	Guard	<code>if (req.session && req.session.user)</code>	API mit gültiger Session	Request zulassen	Zugriff erlaubt	Bestanden
8	Guard	<code>if (req.path.startsWith("/api/"))</code>	API ohne Session	HTTP 401 JSON-Fehler	401 geliefert	Bestanden
9	Workflow	<code>if (payload.costsComplete)</code>	Fall mit <code>costsComplete=true</code>	Statusübergang Team	Status gesetzt/validiert	Bestanden
10	Workflow	<code>if (!hasValidCostsCompletePrerequisites)</code>	Fehlende Pflichtfelder bei Abschluss	HTTP 400 Abbruch	Validierungsfehler ausgegeben	Bestanden
11	Workflow	<code>if (!existingDamageCase)</code>	Update unbekannte ID	HTTP 404	404 geliefert	Bestanden
12	Workflow	<code>if (!rights.canEditCase(...))</code>	Update ohne Bearbeitungsrecht	HTTP 403	Zugriff verweigert	Bestanden
13	Katalog	<code>if (damageCase.costsComplete)</code>	Position bei abgeschlossenem Kostenstatus	HTTP 400/403 blockiert	Änderung blockiert	Bestanden

4. Prüfprotokoll Rechteprüfung (src/rights.js)

ID	Bereich	Verzweigung (<code>if</code>)	Testdurchführung	Erwartetes Ergebnis	Ist-Ergebnis	Status
----	---------	---------------------------------	------------------	---------------------	--------------	--------

ID	Bereich	Verzweigung (if)	Testdurchführung	Erwartetes Ergebnis	Ist-Ergebnis	Status
14	Rollen	<code>if (!roleName visited.has(...))</code>	Rekursive Rollenprüfung	Sauberer Abbruch	Abbruch korrekt	Bestanden
15	Mapping	<code>if (mappingGroupDn && ...)</code>	Exaktes DN-Mapping	Rolle zuordnen	Rolle zugeordnet	Bestanden
16	Mapping	<code>if (mappingGroupName && ...)</code>	Mapping über Gruppenname	Rolle zuordnen	Rolle zugeordnet	Bestanden
17	Mitglied	<code>if (isMember && matchedRoles...)</code>	Ohne Rollenzuordnung	Fallback-Rolle greift	Fallback aktiv	Bestanden
18	Sichtbar	<code>if (hasPermission(..., "view_all"))</code>	Globales Leserecht prüfen	Alle Fälle sichtbar	Alle Fälle sichtbar	Bestanden
19	Leitung	<code>if (hasPermission(...) && status...)</code>	Leitung-Fall Freigabe	Freigabe erlaubt	Freigabe erlaubt	Bestanden

5. Prüfprotokoll Frontend (src/create-damage-case.js)

ID	Bereich	Verzweigung / Testziel	Testdurchführung	Erwartetes Ergebnis	Ist-Ergebnis	Status
20	Kosten	<code>if (!allowed && elements.checked)</code>	costsComplete ohne Recht	Auswahl zurücksetzen	Checkbox zurückgesetzt	Bestanden
21	Pflicht	<code>if (!elements.responsibleParty...)</code>	Verursacher leer gelassen	Feld markiert	Validierung greift	Bestanden
22	API	<code>if (response.status === 401)</code>	Abgelaufene Session	Login-Redirect	Benutzer zum Login geführt	Bestanden
23	Katalog	<code>if (!catalogId)</code>	Ohne Katalogeintrag speichern	Speichern blockiert	Validierung greift	Bestanden
24	Berechnung	Kostenberechnung (1. Versuch)	Katalogposition wählen	Korrekte Summenbildung	Logikfehler bei Berechnung	Nicht bestanden
25	Berechnung	Kostenberechnung (2. Versuch)	Nachtest nach Logik-Fix	Korrekte Summenbildung	Ergebnis korrekt	Bestanden
26	Lookup	KBA-Prüfung (1. Versuch)	Kennzeichen-Eingabe	Zuordnung Kreis/Stadt	Fehlerhafte DB-Abfrage	Nicht bestanden

ID	Bereich	Verzweigung / Testziel	Testdurchführung	Erwartetes Ergebnis	Ist-Ergebnis	Status
27	Lookup	KBA-Prüfung (2. Versuch)	Nachtest nach DB-Fix	Zuordnung Kreis/Stadt	Zuordnung möglich	Bestanden
28	Adresse	<code>if (!elements.city.value.trim())</code>	Stadt im Adressblock leer	Validierungsfehler	Adresse unvollständig	Bestanden

6. Ergebnis

- Alle dokumentierten White-Box-C1-Prüffälle wurden erfolgreich durchgeführt.
- Fehler bei der AD-Konfiguration, der KBA-Zuordnung und der Kostenberechnung wurden iterativ behoben und durch erfolgreiche Nachtests bestätigt.
- Die C1-Überdeckung wurde für die implementierten Kernkomponenten nachgewiesen.

Genehmigter Projektantrag



Prüfung: Sommer 2026

Ausbildungsberuf

Fachinformatiker/Fachinformatikerin Fachrichtung: Anwendungsentwicklung

Timo Funk

Identnummer: 1363017

E-Mail-Adresse: Timo.Funk@lbm.rlp.de | Telefon: 026130291283

Ausbildungsbetrieb: Landesbetrieb Mobilität Rheinland-Pfalz

Betrieblicher Betreuer / Ausbilder: Sascha Konieczny

E-Mail-Adresse: sascha.konieczny@lbm.rlp.de | Telefon: 026130291279

Thema der Projektarbeit

Vorstellung LBM

Der Landesbetrieb Mobilität, kurz LBM, ist die obere Verkehrsaufsichtsbehörde. Der LBM plant, baut und unterhält die Straßen in Rheinland-Pfalz. Dazu gehören die Landes-, Bundesstraßen und mancherorts auch die Kreisstraßen. Der LBM ist dem Ministerium für Wirtschaft, Verkehr, Landwirtschaft und Weinbau, kurz MWVLW, unterstellt. Außerdem plant, unterhält und genehmigt der LBM Radwege und Schienenwege. Ein weiteres Aufgabengebiet gilt dem Luftverkehr. Dort ist der LBM die Luftfahrt- und Luftsicherheitsbehörde für alle Belange der Bürger in Bezug auf die Luftfahrt. Im ganzen Land Rheinland-Pfalz sind etwa 3200 Mitarbeiter beschäftigt, wobei sich ca. 460 Mitarbeiter in der Zentrale in Koblenz befinden. Die anderen Mitarbeiter sind auf acht regionale Dienststellen verteilt, denen wiederum Straßenmeistereien und weitere Liegenschaften nachgeordnet sind.

Vorstellung Fachgruppe Informations- und Telekommunikationstechnik

Die Fachgruppe Informations- und Telekommunikationstechnik befindet sich am Hauptstandort Koblenz und besteht aus ca. 45 Mitarbeitern. An jeder Dienststelle sind darüber hinaus jeweils meist zwei Mitarbeiter für den Fachbereich Informations- und Telekommunikationstechnik im Einsatz, sodass es ungefähr 60 Mitarbeiter sind. Die Fachgruppe Informations- und Telekommunikationstechnik in der Zentrale ist in die Fachteams IT-Service und IT-Organisation, IT-Anwendungen und Verfahren, IT-Hardware und Infrastruktur sowie SAP-Systeme unterteilt. Im Landesbetrieb Mobilität werden derzeit etwa 200 Anwendungen produktiv eingesetzt und noch weitere sind in Testphasen. Diese Vielfalt an Software ist für die tagtäglichen Aufgaben der Straßenmeistereien, oder für die Planung von Straßen, Bauwerken und weiteren komplexen Baustellen erforderlich.

Fachteam 3

Das Fachteam 3 ist für alle fachspezifischen Anwendungen im LBM verantwortlich. Darunter fallen beispielsweise BricsCAD, SaS2 und noch viele weitere. Die Fachadministratoren leisten 2nd-Level-Support, müssen aber auch neue Schnittstellen oder



26.01.2026

auch alternative Software einführen. Dabei muss die Software den Ansprüchen an Behörden entsprechen. Die Software muss in unser System integriert und Schnittstellen müssen konzipiert und implementiert werden, was mit ausführlichen Tests verbunden ist. Von den etwa 200 Anwendungen sind etwa 150 fachspezifisch und werden vom Fachteam 3 betreut.

Entwicklungsumgebung

Entwickelt wird auf einem WSL-Ubuntu 24.04 LTS, welches auf einem Dell Precision 7790 mit einem Intel Core i9 mit 64 GB Arbeitsspeicher läuft. Die Entwicklungsumgebung ist Visual Studio Code. Als Hauptsprache wird JavaScript verwendet. Die Betrieblichen Betreuer sind Armin Schäfer für technische Fragen und Christian Willms für fachliche Fragen.

Thema der Projektarbeit

Es soll die Software Schäden an Straßeneinrichtungen 2 (kurz SaS2) neuentwickelt werden. Da die Software nicht mehr den aktuellen Standards entspricht. Die Dokumentation wird projektbegleitend erstellt und anschließend im LBM eigenen Wiki-System bereitgestellt. Dort werden alle Entwicklerdokumentationen gesammelt und für andere Mitarbeiter der IT zugänglich gemacht.

1 Thema der Projektarbeit

Vorstellung LBM

Der Landesbetrieb Mobilität, kurz LBM, ist die obere Verkehrsaufsichtsbehörde. Der LBM plant, baut und unterhält die Straßen in Rheinland-Pfalz. Dazu gehören die Landes-, Bundesstraßen und mancherorts auch die Kreisstraßen. Der LBM ist dem Ministerium für Wirtschaft, Verkehr, Landwirtschaft und Weinbau, kurz MWVLW, unterstellt. Außerdem plant, unterhält und genehmigt der LBM Radwege und Schienenwege. Ein weiteres Aufgabengebiet gilt dem Luftverkehr. Dort ist der LBM die Luftfahrt- und Luftsicherheitsbehörde für alle Belange der Bürger in Bezug auf die Luftfahrt. Im ganzen Land Rheinland-Pfalz sind etwa 3200 Mitarbeiter beschäftigt, wobei sich ca. 460 Mitarbeiter in der Zentrale in Koblenz befinden. Die anderen Mitarbeiter sind auf acht regionale Dienststellen verteilt, denen wiederum Straßenmeistereien und weitere Liegenschaften nachgeordnet sind.

Vorstellung Fachgruppe Informations- und Telekommunikationstechnik

Die Fachgruppe Informations- und Telekommunikationstechnik befindet sich am Hauptstandort Koblenz und besteht aus ca. 45 Mitarbeitern. An jeder Dienststelle sind darüber hinaus jeweils meist zwei Mitarbeiter für den Fachbereich Informations- und Telekommunikationstechnik im Einsatz, sodass es ungefähr 60 Mitarbeiter sind. Die Fachgruppe Informations- und Telekommunikationstechnik in der Zentrale ist in die Fachteams IT-Service und IT-Organisation, IT-Anwendungen und Verfahren, IT-Hardware und Infrastruktur sowie SAP-Systeme unterteilt. Im Landesbetrieb Mobilität werden derzeit etwa 200 Anwendungen produktiv eingesetzt und noch weitere sind in Testphasen. Diese Vielfalt an Software ist für die tagtäglichen Aufgaben der Straßenmeistereien, oder für die Planung von Straßen, Bauwerken und weiteren komplexen Baustellen erforderlich.

Fachteam 3

Das Fachteam 3 ist für alle fachspezifischen Anwendungen im LBM verantwortlich. Darunter fallen beispielsweise BricsCAD, SaS2 und noch viele weitere. Die Fachadministratoren leisten 2nd-Level-Support, müssen aber auch neue Schnittstellen oder auch alternative Software einführen. Dabei muss die Software den Ansprüchen an Behörden entsprechen. Die Software muss in unser System integriert und Schnittstellen müssen konzipiert und implementiert werden, was mit ausführlichen Tests verbunden ist. Von den etwa 200 Anwendungen sind etwa 150 fachspezifisch und werden vom Fachteam 3 betreut.

Entwicklungsumgebung

Entwickelt wird auf einem WSL-Ubuntu 24.04 LTS, welches auf einem Dell Precision 7790 mit einem Intel Core i9 mit 64 GB Arbeitsspeicher läuft. Die Entwicklungsumgebung ist Visual Studio Code. Als Hauptsprache wird JavaScript verwendet. Die Betrieblichen Betreuer sind Armin Schäfer für technische Fragen und Christian Willms für fachliche Fragen.

Thema der Projektarbeit

Es soll die Software Schäden an Straßeneinrichtungen 2 (kurz SaS2) neuentwickelt werden. Da die Software nicht mehr den aktuellen Standards entspricht. Die Dokumentation wird projektbegleitend erstellt und anschließend im LBM eigenen Wiki-System bereitgestellt. Dort werden alle Entwicklerdokumentationen gesammelt und für andere Mitarbeiter der IT zugänglich gemacht.

2 Geplanter Bearbeitungszeitraum

Die Bearbeitung des Projekts ist für den Zeitraum vom 01.03.2026 bis zum 22.04.2026 geplant.

3 Ausgangssituation

Die Software Sas2 dient der Erfassung aller Schäden an Straßeneinrichtungen. Dort werden alle Schäden, die an jeglichen Straßeneinrichtungen geschehen, dokumentiert. Auch werden durch diese Software die jeweiligen Rechnungen und Bescheide an die Verursacher versendet. Nach Beendigung der Erfassung und nachdem die Rechnung versendet wurde, wird der Schadensfall an SAP weitergeleitet, damit der Zahlungseingang korrekt gebucht werden kann. Falls notwendig, werden in der Software Beweise gesammelt und gespeichert, um diese für spätere Rückfragen schon bereit liegen zu haben. Es gibt insgesamt drei wichtige Bereiche innerhalb der Software. Dies ist die Erfassung des Schadens, der von den einzelnen Straßenmeistereien gemacht wird. Nachdem die Schadenserfassung vollständig abgeschlossen wurde, wird der Fall an das Team der Schadensabteilung weitergeleitet, welche dann die Rechnungen schreiben. Beim Schreiben der Rechnungen gilt das Vier-Augen-Prinzip, so dass die Fachgruppenleitung den Fall als korrekt bestätigen muss. Wenn die Rechnungen geschrieben wurden, wird der Fall an das Finanzteam weitergeleitet, für die Zuordnung der Beträge. Allerdings ist die Software schon veraltet und muss an die neuen Anforderungen des LBM angepasst werden.

4 Projektziel

Das Ziel des Projektes ist die Neuentwicklung von einzelnen Bausteinen der Software für eine spätere komplette Eigenentwicklung. Es sollen im Projekt die Bausteine des Anmeldescreens mit eingebauter Rechteprüfung, der Eingabemaske für die Straßenmeistereien inklusive dem

Dateiupload, und die Funktion sich als Schadensteammitarbeiter die einzelnen Fälle anzusehen realisiert werden. Dies beinhaltet die Anpassung des Designs an das Corporate Design des LBM, des Weiteren wird auch eine Verbesserung der Workflows angestrebt. Als Produktumgebung wird es als interne Webseite erreichbar sein.

5 Zeitplanung

1. Analyse

1.1. Anforderungsanalyse mit Auftraggeber (2h)

1.2. Ist-Analyse (1h)

1.3. Soll-Konzept erarbeiten (2h)

2. Planung

2.1. Rücksprache mit Auftraggeber (1h)

2.2. Erstellung verschiedener Diagramme (Use-Case, UML, Ablauf-Diagramme) (2h)

3. Programmieren

3.1. Projekt anlegen mit Projektstruktur (4h)

3.2. Datenbankdesign erstellen (4h)

3.3. Datenbank befüllen (4h)

3.4. Anmeldescreen erstellen (2h)

3.5. Funktion zur Rechteprüfung erstellen (6h)

3.6. Erstellen der Funktion und Eingabemaske für Straßenmeistereien (8h)

3.7. Erstellung einer Funktion, für Datei Uploads und Speicherung (3h)

3.8. Erstellen der Funktion und Eingabemaske Schadensteammitarbeiter (7h)

4. Testen

4.1. Testen des Anmeldescreens und Rechteprüfung (4h)

4.2. Testen der Funktionen für Dateiuploads und der Eingabemaske (8h)

4.3. Abschlusstest aller Funktionen (8h)

5. Dokumentation schreiben

5.1. Anwenderdokumentation (4h)

5.2. Entwicklerdokumentation (8h)

5.3. Betriebsinterne Dokumentation (2h)

6 Anlagen

Projektantrag in PDF-Form

7 Präsentationsmittel

Das Projekt wird mit Hilfe einer Präsentation präsentiert.

8 Hinweis

Ich bestätige, dass der Antrag „Betrieblicher Auftrag“ dem Ausbildungsbetrieb vorgelegt und vom Auszubildenden genehmigt wurde. Der Antrag „Betrieblicher Auftrag“ enthält keine Betriebsgeheimnisse. Soweit diese für die Antragstellung notwendig sind, wurden nach Rücksprache mit dem Auszubildenden die entsprechenden Stellen unkenntlich gemacht. Ich bestätige weiterhin, dass der Antrag "Betrieblicher Auftrag" eigenständig von mir angefertigt wurde. Ferner sichere ich zu, dass im Antrag personenbezogene Daten (d.h. Daten über die eine Person identifizierbar oder



26.01.2026

bestimmbar ist) nur verwendet werden, wenn die betroffene Person hierin eingewilligt hat. Ich wurde darauf hingewiesen, dass meine Arbeit bei Täuschungshandlungen bzw. Ordnungsverstößen mit „null“ Punkten bewertet werden kann. Ich bin weiter darüber aufgeklärt worden, dass dies auch dann gilt, wenn festgestellt wird, dass meine Arbeit im Ganzen oder zu Teilen mit der anderer Prüfungsteilnehmer übereinstimmt.

Abbildung 23: Genehmigter Projektantrag